



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Милица Ђумић

TestDataGen: Језик специфичан за домен генерисања тестних података

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Милица Ђумић	Број индекса:	R2 5/2022
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

TestDataGen: Језик специфичан за домен генерисања тестних података

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - мастер рад
Аутор, АУ:	Милица Ђумић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	TestDataGen: Језик специфичан за домен генерисања тестних података
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	7/69/12/1/25/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Језици специфични за домен, генератори тестних података,
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Научни рад описује софтверски алат за паметно генерисање тестних података за различите типове излазних података.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.09.2026.
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Milica Đumić
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	TestDataGen: A Domain-Specific Language for Test Data Generation
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/69/12/1/25/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Domain specific language, test data generator,
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	Scientific paper describes a software tool for the smart generation of test data for various types of output data .
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.09.2026.
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Језици специфични за домен	3
2.1	Апстрактна синтакса	4
2.2	Конкретна синтакса	4
2.3	Семантика	5
2.4	Архитектура	5
3	Спецификација <i>TestDataGen</i> система	7
3.1	Апстрактна синтакса	8
3.2	Конкретна синтакса	10
3.3	Семантика	17
4	Архитектура језика <i>TestDataGen</i>	21
4.1	Основни ток обраде	21
4.2	Интерфејс командне линије	23
4.3	Парсирање и модел језика	23
4.4	Семантичка валидација	23
4.5	Генерисање тестних вредности	24
4.6	Комбиновање вредности	24
4.7	Обрада референци	24
4.8	Генератори излазних формата	25
4.9	Коришћене технологије	25
4.10	Битне карактеристике архитектуре	26
5	Разлози избора излазних формата	27
5.1	<i>SQL</i> формат	27
5.2	<i>JSON</i> формат	27
5.3	<i>HTML</i> извештај о покривености	28
5.4	<i>CSV</i> формат	29
5.5	<i>YAML</i> формат	30
5.6	<i>Neo4j</i> формат	30
5.7	Значај подршке више формата	31
6	Имплементација додатних генератора излазних формата	33
6.1	<i>CSV</i> генератор	33
6.2	<i>YAML</i> генератор	37
6.3	<i>Neo4j</i> генератор	39
7	Закључак	55
	Списак слика	57
	Списак листинга	59
	Списак табела	61
	Списак коришћених скраћеница	63
	Списак коришћених појмова	65
	Биографија	67
	Литература	69

Језик специфичан за домен (ЈСД, енг. *Domain-Specific Language*) представља језик прилагођен описивању доменских проблема концептима карактеристичним за одређен домен. За разлику од језика опште намене (ЈОН, енг. *General-Purpose Languages*), ЈСД омогућава да се проблеми из одређене области опишу на начин који је ближи самом домену, чиме се поједностављује њихово моделовање и омогућава аутоматска обрада тако дефинисаних спецификација.

TestDataGen је систем заснован на језику специфичном за домен генерисања тестних података. У оквиру система корисник помоћу *TestDataGen* спецификације описује структуру тестних података, њихова ограничења и стратегије генерисања. Након тога, систем на основу такве спецификације генерише тестне податке. Добијени подаци могу бити представљени у различитим форматима, у зависности од система или алата који ће се користити за њихову даљу обраду.

У оквиру овог мастер рада, систем *TestDataGen* проширен је са три додатна генератора излазних формата, а то су: *CSV*, *YAML* и *Neo4j*. Прва два генератора омогућавају представљање генерисаних тестних података у табеларном и хијерархијски организованом текстуалном формату. *Neo4j* генератор уводи могућност представљања тестних података у облику графа. За потребе имплементације *Neo4j* генератора, систем је проширен конструкцијом *relationship*, којом се омогућава описивање повезаности између ентитета. На основу овакве спецификације могуће је генерисати чворове и везе између њих у графној бази података *Neo4j*. Подржане су, такође, различите кардиналности веза између чворова у *Neo4j* бази.

Проширење система *Neo4j* генератором зато не представља само увођење новог излазног формата, већ обухвата и проширење језика, заједничког механизма генерисања и развојног окружења за рад са дефинисаним спецификацијама. У оквиру рада додата је подршка за дефинисање и обраду веза, имплементирани су одговарајуће стратегије њиховог генерисања, а функционалности *Visual Studio Code* екстензије проширене су како би подржале нове елементе језика. На тај начин проширена је, не само могућност представљања резултата, већ и изражајност самог *TestDataGen* језика.

Глава 2

Језици специфични за домен

Софтверски језици [1] су подгрупа језика чија је намена да омогуће комуникацију човек-рачунар или рачунар-рачунар. Веома важна особина софтверски језика јесте да су људима лаки за разумевање, а истовремено да се ефикасно процесуирају на рачунару. Њих можемо поделити на перспективне и декриптивне. Перспективни софтверски језици описују будуће системе, а декриптивни већ постојеће.

Језици опште намене (ЈОН, енг. *General-Purpose Languages*) [1] су подгрупа софтверских језика. Њихова примена може да буде у различитим областима и за креирање различитих софтверских апликација. Зависно од проблема који се решава, неки ЈОН може да буде погоднији од неког другог, иако су теоријски сви ЈОН у могућности да опишу решења из многобројних домена. Пример тога би био софтверски језик C погодан за писање системских програма ближих хардверу.

Домен [1] се односи на сферу у којој делујемо. Сваки домен садржи концепте и њихове међусобне везе. Познавање концепата омогућава доменским експертима да се боље изразе, лакше и брже споразумеју међусобно. Један домен може да се састоји од више поддомена. Пример домена може бити банкарство и поддомени би били банкарство са становништвом, банкарство са привредом и приватно банкарство. Познавање домена проблема помаже у лакшем и тачнијем решавању.

Језици специфични за домен (ЈСД, енг. *Domain-Specific Languages*) [1], [2] су подврста софтверских језика специјализована за један узак домен. На тај начин ЈСД јасније и прецизније описује решење постављеног проблема. Такође, употреба концепата из домена омогућава доменским експертима да лакше мапирају концептуална решења у програмски код. Са друге стране, због своје специјализације, ЈСД није намењен за решавање проблема ван домена за који је дефинисан. То указује да је примена ЈСД-а мањег обима него примена ЈОН-а.

Као што је речено, ЈСД омогућавају да се решење опише јасније јер користе концепте из домена којем припада проблем који се решава. На тај начин се доменским експертима умањује време потребно да се концептуално решење премапира на програмски код. Због употребе вишег нивоа апстракције, решења написана у ЈСД често захтевају мањи број линија кода што може смањити густину софтверске грешке (број софтверских грешака на 1000 линија кода). Искazi су самодокументујући, а уколико постоји потреба да се нешто измени у тумачењу исказа, сами искази неће захтевати измене. Такве промене у обради исказа, захтевају промену постојећег компајлера. На тај начин се посао програмера и доменских експерата раздваја, а и олакшава се одржавање. Такође, дуговечност употребљених исказа повећава дуговечност написаног решења.

Споменуте су неке предности ЈСД-а, али постоје и проблеми које треба споменути. Да би се развио ЈСД, неопходно је да програмери имају јасну слику и добро знање о домену. Да би се то омогућило програмери морају да имају интеракцију са доменским експертима. На тај начин је једино могуће направити ефикасан ЈСД. То захтева време и повећава цену и развоја и одржавања ЈСД-а. Поред тога, додатни напор је потребан за развијање алата који обрађују ЈСД (то су по потреби парсери, интерпретери, компајлери и генератори кода). Ипак када се дефинише и имплементира добар ЈСД, ефикасност у решавању проблема се повећава многоструко. Још једна мана увођења нових ЈСД-а јесте језичка какофонија (енг. *language cacophony*) што значи да постоји потреба да програмери познају велики број софтверских језика.

Примери ЈСД-а:

- SQL (енг. *Structured Query Language*) је специјализован за рад са релационим базама података
- HTML (енг. *HyperText Markup Language*) дефинише садржај веб страница

- CSS (енг. *Cascading Style Sheets*) служи за стилизовање садржаја веб страница.

Сваки софтверски језик, па тако и ЈСД, има следеће градивне елементе:

- апстрактну ситаксу
- једну или више конкретних синтакси
- семантику.

2.1 Апстрактна синтакса

Апстрактна синтакса [1] описује концепте из домена, њихове међусобне везе и особине. Омогућава да се одреди да ли је исказ валидан са становишта структуре, али не узима у обзир употребљену конкретну синтаксу (објашњену у наредном потпоглављу). Такође, апстрактна синтакса нема појавни облик. Да би се дефинисала, често се користи језик погодан за опис структуре (на пример UML дијаграм класа [3]). Уколико се успешно дефинише, апстрактна синтакса описује валидне структуре исказа у том ЈСД-у јер узимајући у обзир битне концепте домена занемарује имплементационе детаље и начин записа исказа.

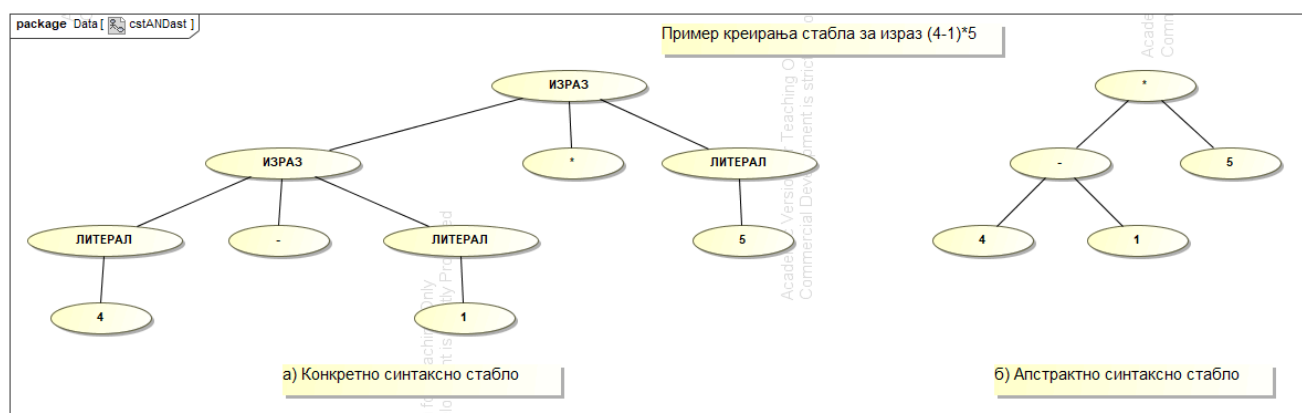
Апстрактно синтактно стабло (AST, енгл. *Abstract Syntax Tree*) [1] јесте апстрактна структурна репрезентација конкретног исказа. Чворови стабла дефинишу конкретне инстанце концепата, а везе њихове структурне односе. На тај начин се занемарују имплементациони детаљи. На пример, исти исказ у префиксној, инфиксној и постфиксној нотацији може да резултује истим апстрактним синтаксним стаблом јер обе нотације описују исту суштину исказа.

2.2 Конкретна синтакса

Конкретна синтакса (или нотација) [1] језика представља интерфејс према крајњем кориснику. У суштини, дефинише се на који начин корисник записује исказе неког ЈСД-а. Најчешће конкретне синтаксе су текстуална и графичка, али поред њих постоје и табеларна, типа стабла и многе друге. Због великог избора, дешава се да један ЈСД има више различитих конкретних синтакси. На тај начин, избор најпогодније конкретне синтаксе зависи од проблема који се решава. Претходно наведени пример са префиксном, инфиксном и постфиксном нотацијом илуструје постојање више конкретних синтакси које описују суштински исти исказ.

Зависно од употребљене конкретне синтаксе, написани исказ ЈСД-а ће моћи да се мапира на тачно једно конкретно синтаксно стабло (CST, енгл. *Concrete Syntax Tree*). Као што је горе наведено у примеру, више CST-а могу да се мапирају на једно AST уколико је смисао исказа идентичан. То омогућава да се сваки проблем решава употребом конкретне синтаксе која је најпогоднија у том случају.

Пример разлике конкретног и апстрактног синтаксног стабла је дат на слици 1.



Слика 1: Пример разлике конкретног и апстрактног синтаксног стабла.

Такође, фактор за одабир конкретне синтаксе може да буде крајњи корисник. Уколико је група крајњих корисника приврженија графичкој нотацији, употребу ЈСД-а ће олакшати ако је дефинисана графичка кон-

кретна синтакса за тај ЈСД. У сваком случају, крајњем кориснику може бити остављена одређена слобода у начину представљања исказа, то је дефинисано појмом секундарне нотације. Секундарна нотација утиче на лакоћу читања. Једноставно ју је објаснити на примеру графичких конкретних синтакси где се односи на величину, боју елемената и начин повезивања графичких елемената. Поред тога, код текстуалних конкретних синтакси се може применити на употребу празних карактера (енг. *white spaces*). Неки језици, попут Python-а, прописују правила употребе празних карактера у сврху угњеждавања и самим тим елиминишу проблеме у читљивости.

2.3 Семантика

Семантика [1] дефинише смисао исправног језичког исказа. Можемо је дефинисати на различите начине. Један од начина јесте да се реализује превођењем семантички валидног исказа ЈСД-а у исказ на ЈОН-у. Начин превођења може бити компајлирање или интерпретирање. Код компајлирања се исказ преводи пре извршавања, док код интерпретирања у току обраде се исказ анализира и преводи. То се користи јер ЈОН већ има дефинисану семантику (могуће га је превести или интерпретирати у машински код).

2.4 Архитектуре

Приликом имплементације ЈСД-а, као што је горе поменуто, можемо омогућити превођење исказа ЈСД у исказе неког другог софтверског језика. Архитектура те подршке за ЈСД [1] може бити базирана на компајлерима или интерпретерима. У случају архитектуре базиране на компајлерима, искази ЈСД се превode на исказе другог језика пре покретања програма. Најчешће се превode на софтверски језик са дефинисаном семантиком и постојећим компајлером. Са друге стране, архитектуре базиране на интерпретерима превode исказе ЈСД у време извршавања програма, при чему се тада врши и анализа исправности исказа.

Глава 3

Спецификација *TestDataGen* система

У претходном поглављу описани су језици специфични за домен и њихове основне карактеристике. ЈСД омогућавају мапирање концептуалног решења помоћу концепата који су блиски самом домену. На тај начин се сложени имплементациони детаљи могу сакрити апстракцијом разумљивој корисницима и доменским експертима.

За тестирање различитих софтверских система често је неопходно припремити велику количину података. Ручно креирање таквих података је временски захтевно. Додатан напор захтева уколико је потребно тестирати велики број комбинација вредности или специфичне случајеве. Неки од примера би били граничне или недостајуће вредности. Генерисање случајних вредности је погодно за брзо креирање велике количине података, али не решава проблем са одређеним вредностима које треба уврстити.

Предмет мастер рада јесте проширење језика специфичног за домен под називом *TestDataGen*. Његова намена је дефинисање и аутоматско генерисање тестних података. Основна идеја је омогућавање кориснику да на једноставан начин опише:

- структуру података које жели да генерише
- ограничења над подацима
- стратегије генерисања и
- међусобне односе.

На основу тако дефинисане спецификације, систем генерише скуп тестних података у једном или више различитих формата.

Мана генерисања искључиво случајним избором се превазилази тако што корисник изабере стратегију на основу које се генеришу подаци. Подржане су стратегије засноване на анализи граничних вредности, партицијама еквиваленције и генерисању специјалних случајева. Поред тога, ЈСД омогућавају дефинисање начина на који се вредности различитих поља међусобно комбинују, како би се избегла комбинаторна експлозија броја генерисаних тестних података.

Корисник не описује поступак генерисања података корак по корак, него дефинише особине жељених података и правила која генератор треба да примени. Овај приступ се назива декларативним.

Основни концепти језика су шема, ентитет, поље и веза. У оквиру шеме (енг. *schema*) дефинишу се глобална подешавања. Ентитети (енг. *entity*) представљају структуре за које се генеришу подаци, а везе (енг. *relationship*) дефинишу односе између ентитета. Сваки ентитет садржи поља (енг. *field*) са одговарајућим типовима и ограничењима. У наставку је наведен пример структуре ентитета.

```
entity User {
  fields {
    id: uuid
    email: email {
      unique,
      include [null, empty, invalid]
    }
    username: username {
      unique
    }
    age: number {
      range 18..99,
```

```
        boundary,
        partitions [27, 67]
    }
    status: enum ["active", "suspended", "pending"] {
        coverage 100
    }
}
config {
    generate: 50
    combination_strategy: each-used
    include: [
        { id: "00000000-0000-0000-0000-000000000001", email: "admin@system.local",
username: "superadmin", age: 30, status: "active" },
        { id: "00000000-0000-0000-0000-000000000002", email: null, username:
"guest_user", age: 17, status: "pending" }
    ]
}
```

На основу овакве спецификације није потребно ручно имплементирати алгоритме сваког појединачног тестног случаја. Корисник опише домен података, њихове типове и ограничења, док *TestDataGen* на основу дефинисаних правила одреди вредности које треба генерисати. На пример за поље *age* могу бити дефинисане вредности из траженог опсега, вредности непосредно изнад и испод граница и представници различитих партиција еквиваленције.

Поред аутоматски генерисаних података, *TestDataGen* омогућава и употребу експлицитно наведених вредности. Тиме кориснику даје могућност да укључи конкретне вредности које морају бити укључене у генерисани скуп података. То је нарочито корисно за тестирање познатих грешака и регресионо тестирање.

Важан аспект проблема који *TestDataGen* решава представља и број могућих комбинација вредности. Уколико сваки атрибут ентитета може да има више вредности, Декартов производ скупа могућих вредности може довести до експанзије у броју генерисаних тестних случајева. Због тога језик подржава различите стратегије комбинације вредности и то су потпуно комбиновање (Декартов производ могућности), комбинација парова (сваки пар да се појави једном) и да се свака генерисана вредност појави најмање једном. На тај начин корисник има могућност да одабере компромис између жељене покривености тестних података и броја генерисаних података.

Коначан резултат обраде *TestDataGen* спецификације може бити генерисан у једном или више формата. Подржани су следећи формати за генерисање тестних података: *SQL*, *JSON*, *CSV*, *YAML* и *Neo4j* (он генерише *Cypher* скрипту за рад са графном базом). Поред самих тестних података, систем може да генерише *HTML* извештај о покривености генерисаних тестних случајева.

На овај начин *TestDataGen* раздваја опис података и стратегије њиховог генерисања од конкретне имплементације генератора. Корисник на вишем нивоу апстракције дефинише шта је потребно тестирати, а систем генерише одговарајуће тестне податке. Тиме се примењују принципи језика специфичних за домен у области аутоматског генерисања тестних података.

У наставку ће бити описани основни градивни елементи језика *TestDataGen*, његова апстрактна и конкретна синтакса, семантика и архитектура система. Након тога ће бити детаљније представљене стратегије које се користе за паметни избор тестних података, као и имплементирани генератори излазних формата.

3.1 Апстрактна синтакса

Апстрактна синтакса језика *TestDataGen* дефинише концепте домена генерисања тестних података, њихове особине и међусобне односе. Није дефинисано на који начин се концепти записују, већ само структура сваке валидно написане спецификације у језику.

Почетни концепт апстрактне синтаксе је шема (енг. *schema*). Једна шема представља целокупну спецификацију генерисаних тестних података. Шема има назив, а може да садржи опис, почетну вредност за генерисање случајних бројева, подразумевану стратегију генерисања и подразумевану стратегију комбиновања вредности. Осим тога, шема садржи скуп елемената, који могу бити ентитети или везе. На тај начин, модел омогућава опис самих структура података и њихових међусобних односа.

Ентитет (енг. *entity*) представља структуру за коју се генеришу тестни подаци. Сваки ентитет има назив и скуп поља. Поред структуре података, ентитет може да садржи и конфигурацију генерисања. Конфигурација омогућава одређивање броја података који треба да буду генерисани, стратегије комбиновања вредности и експлицитно дефинисање тестних случајева.

Поље (енг. *field*) представља појединачну особину ентитета. Свако поље има назив и тип. Тип поља може да буде један од уграђених типова података, набројиви тип или референца на други ентитет.

Уграђени типови обухватају различите типове података који се могу генерисати. То су: идентификатори, адресе електронске поште, имена, презимена, корисничка имена, бројеви, логичке вредности, датуми и други типови података подржани генератором. Набројиви тип (енг. *enum*) омогућава дефинисање коначног скупа могућих вредности поља. Референтни тип (енг. *ref*) успоставља структурну зависност између ентитета, тако што се поље повезује са другим ентитетом. Референца може представљати појединачну вредност или колекцију вредности, за коју је могуће дефинисати минималан и максималан број елемената.

Поред типа, поље може да има скуп ограничења (енг. *constraints*). Ограничења дефинишу скуп правила која утичу на избор и генерисање вредности. Подржана су ограничења за дефинисање опсега вредности, анализу граничних случајева, партиционисање домена, јединственост вредности, прецизност нумеричких вредности, специјалне вредности, покривеност и укључивање вредности као што су *null*, празна или неважећа вредност. Такође је могуће дефинисати посебну стратегију генерисања за појединачно поље.

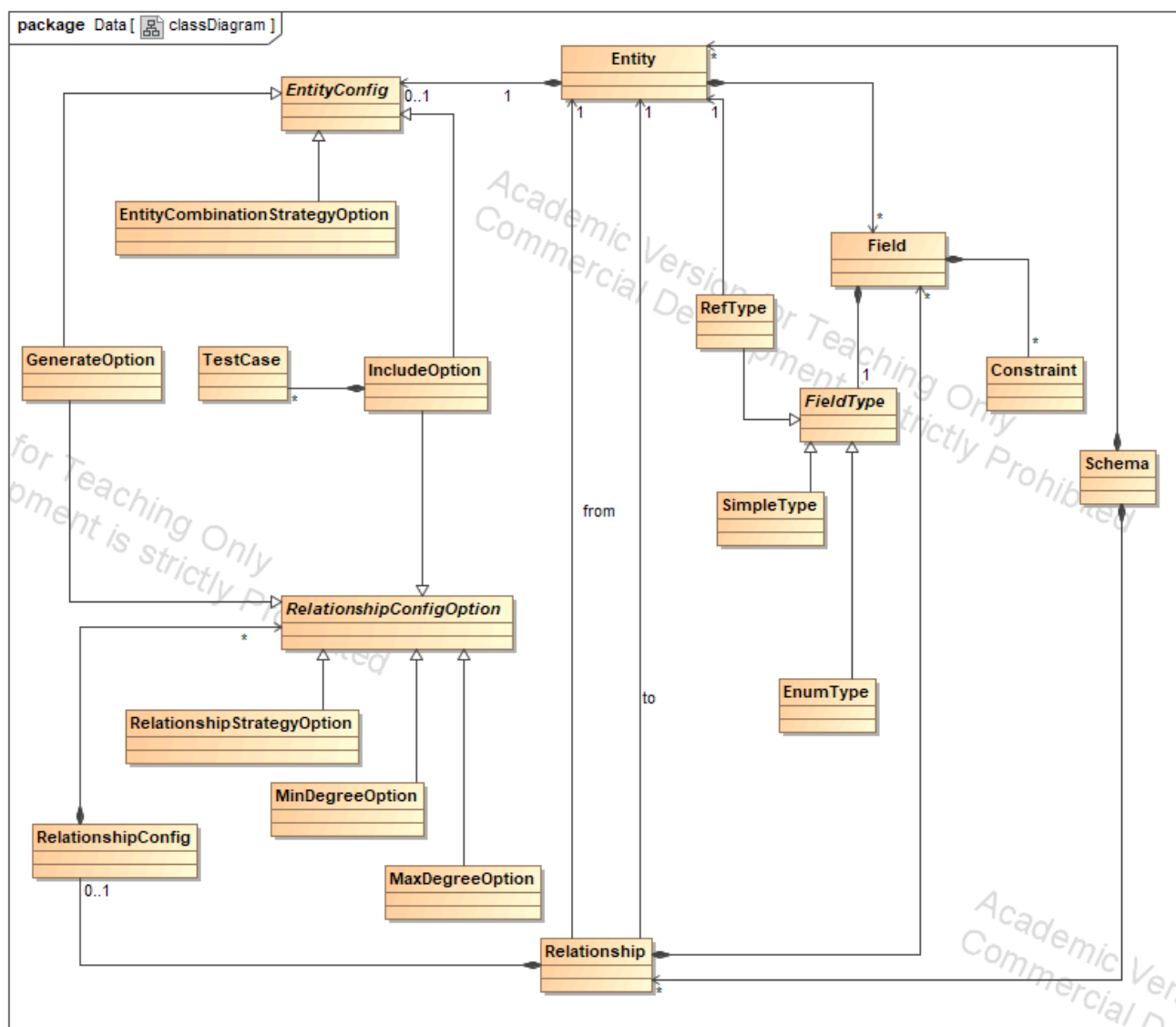
Други основни елемент шеме је веза (енг. *relationship*). Веза описује однос између два ентитета. Свака веза има назив, полазни ентитет (енг. *from*) и циљни ентитет (енг. *to*). Поред тога, веза може да садржи сопствена својства. Својства веза су представљена пољима (као код ентитета) са својим типовима и ограничењима.

За везу је могуће дефинисати конфигурацију. Она одређује начин генерисања односа између инстанци ентитета. Подржане су стратегије један-према-један (енг. *one-to-one*), један-према-више (енг. *one-to-many*), више-према-један (енг. *many-to-one*), више-према-више (енг. *many-to-many*). Конфигурација везе може да садржи и максималан број веза које генератор може да произведе, ограничења минималног и максималног степена повезаности, као и експлицитно дефинисање тестних случајева.

У тренутној имплементацији, дефиниције везе се користе само приликом генерисања *Cypher* скрипти за *Neo4j* графну базу података. Остали генератори обрађују структуре дефинисане ентитетима.

Експлицитно дефинисање тестних случајева представља засебан концепт апстрактне синтаксе. Они се састоје од скупа додела вредности пољима. Вредност може бити скаларна вредност, листа вредности или *null*. На тај начин корисник, поред аутоматски генерисаних вредности, може да дефинише конкретне случајеве који морају бити укључени у резултат генерисања.

Однос између основних концепата апстрактне синтаксе је приказан на слици 2.

Слика 2: Апстрактна синтакса језика *TestDataGen*.

На основу дефинисане граматике, свака *TestDataGen* спецификација се парсира у објектни модел који садржи информације неопходне за њену даљу обраду. Конкретна синтакса језика одређује начин на који корисник записује ентитете, поља, ограничења, стратегије и остале конструкције језика. Алат *textX* [4] на основу граматике формира објекте модела који одговарају конструкцијама дефинисаним у спецификацији. Добијени модел представља основу за даљу семантичку проверу и генерисање тестних података. Над моделом се примењују правила за проверу исправности спецификације, избор вредности и њихово комбиновање, након чега се генерисани подаци прослеђују одговарајућим генераторима излазних формата.

3.2 Конкретна синтакса

Конкретна синтакса језика *TestDataGen* је текстуална. Спецификација се записује у облику текстуалног фајла са екстензијом *.tda* или *.tdg*. Концепти апстрактне синтаксе се представљају помоћу кључних речи, идентификатора и литерала.

Кључне речи одређују структуру спецификације и имају унапред дефинисано значење. То су *schema*, *entity*, *fields*, *config* и *relationship*. Идентификатори се користе за именовање шема, ентитета и поља. Литерали се користе за запис конкретних вредности, као што су цели и реални бројеви, текстуалне и логичке вредности.

Граматика конкретне синтаксе дефинисана је коришћењем алата `textX` [4]. На основу дефинисане граматике, текстуална спецификација се парсира и претвара у објектни модел. `textX` на основу правила граматике креира објекте који представљају конструкте спецификације и њихове везе. Тако добијени модел се даље користи у процесу валидације и генерисања тестних података.

3.2.1 Шема

Основни елемент конкретне синтаксе је шема. Шема се дефинише кључном речју *schema*. Након тога следи њен назив и тело шеме унутар витичастих заграда. Унутар шеме могу се дефинисати глобална подешавања, ентитети и везе између ентитета. Глобална подешавања су опциона и обухватају опис шеме, почетну вредност генератора случајних бројева, подразумевану стратегију генератора и подразумевану стратегију комбиновања вредности.

```
schema ComprehensiveSystem {
  description: "Master rad"
  seed: 98765
  strategy: smart
  combination_strategy: pairwise
}
```

Атрибут *description* омогућава додавање текстуалног описа шеме. Помоћу атрибута *seed* задаје се почетна вредност генератора случајних бројева. Атрибутима *strategy* и *combination_strategy* дефинишу се подразумеване стратегије генерисања и стратегије комбиновања вредности. Пошто су наведена подешавања опциона, шема може садржати само назив и елементе који се у њој дефинишу.

3.2.2 Ентитет

Ентитети се у конкретној синтакси дефинишу помоћу кључне речи *entity*. Након тога следи назив ентитета и његово тело. Тело ентитета садржи блок *fields*. У њему се дефинишу поља. Опционо се може навести и блок *config* са подешавањима генератора.

Поље се записује у облику:

```
name: type
```

где *name* представља назив поља, а *type* његов тип. Уколико поље има ограничења, она се наводе унутар витичастих заграда након типа.

```
entity Profile {
  fields {
    id: uuid
    bio: string
    birth_date: date {
      range "1950-01-01".."2005-12-31",
      boundary
    }
    is_verified: boolean {
      coverage 100
    }
  }
  config {
    generate: 30
  }
}
```

У наведеном примеру дефинисан је ентитет *Profile* са четири поља. Поље *id* има тип *uuid*. Поље *bio* има тип *string*. Поље *birth_date* има тип *date*, а за њега је дефинисан опсег вредности и да се врши анализа граничних случајева. Поље *is_verified* има тип *boolean*, а за њега је дефинисана покривеност. Такође, дефинисана

је конфигурација за ентитет са задатим бројем генерисаних инстанци ентитета. Објашњења подржаних ограничења ће бити описана у наредним поглављима.

3.2.3 Типови поља

Тип поља може припадати једној од три категорије:

- једноставни тип
- набројиви тип
- референтни тип.

Једноставни типови представљају унапред дефинисане типове података које генератор подржава. У њих спадају: *uuid*, *email*, *firstName*, *lastName*, *fullName*, *number*, *boolean*, *datetime*, *date*, *string*, *productName*, *companyName*, *address*, *city*, *country*, *phone*, *url* и *username*.

Набројиви типови омогућавају да се скуп могућих вредности поља експлицитно наведе. Дефинишу се кључном речју *enum*. Након тога се у угластим заградама наводе могуће вредности.

```
status: enum ["active", "suspended", "pending"]
```

На овај начин генератор приликом избора вредности за поље *status* користи само вредности наведене у дефиницији набројивог типа.

Референтни тип се користи за успостављање везе између поља једног ентитета и другог ентитета. Дефинише се кључном речју *ref*. Након тога следи назив ентитета на који се референца односи.

```
user: ref User
```

Референца може представљати и колекцију вредности. У том случају се након назива ентитета наводе угласте заграде. За колекцију је могуће дефинисати минималан и максималан број ентитета.

```
users: ref User[] count 1..5
```

Опционим кључним речима *count* и *boundary* могу се додатно дефинисати правила за генерисање броја референци.

3.2.4 Ограничења над пољима

Поред типа, пољу се могу придружити ограничења која утичу на начин избора и генерисања њихових вредности. Ограничења се наводе унутар витичастих заграда након типа поља.

TestDataGen подржава више врста ограничења. Ограничењем *range* дефинише се опсег дозвољених вредности. Ограничење *boundary* омогућава укључивање граничних случајева. Ограничење *partition* омогућава одређивање на колико партиција ће бити подељен домен. Помоћу *partitions* се дефинишу представници партиција. Ограничење *unique* захтева јединственост генерисаних вредности за поље. За нумеричке вредности може се дефинисати и прецизност помоћу ограничења *precision*.

```
age: number {  
  range 18..99,  
  boundary,  
  partitions [27, 67]  
}
```

Поред горенаведених ограничења, могуће је дефинисати специјалне вредности помоћу *special*. Кључном речју *include*, омогућено је захтевати укључивање посебних вредности као што су *null*, празна вредност и неважећа вредност. Захтевани проценат покривености дефинише се помоћу *coverage*.

```
price: number {  
  range 0..500.0,  
  boundary,  
  precision 2,  
}
```

```
    special ["0", "99.99", "499.99"]
  }
```

За појединачно поље може се дефинисати и стратегија генерисања помоћу ограничења *strategy*. На тај начин је могуће одступати од подразумеване стратегије дефинисања на нивоу шеме или ентитета.

3.2.5 Конфигурација генерисања

Конфигурација ентитета дефинише начин генерисања инстанци ентитета. Наводи се унутар блока *config*. Основна опција је *generate*, којом се одређује број података који треба генерисати.

```
entity Course {
  fields {
    id: uuid
    title: productName
    price: number {
      range 0..500.0,
      boundary,
      precision 2,
      special ["0", "99.99", "499.99"]
    }
    capacity: number {
      range 10..200,
      partition 5
    }
  }
  config {
    generate: 20
    combination_strategy: full
  }
}
```

Поред броја генерисаних инстанци, на нивоу ентитета могуће је дефинисати стратегију комбиновања вредности помоћу *combination_strategy*. Такође је могуће експлицитно навести тестне случајеве који морају бити укључени у резултат генерисања. Оваква организација омогућава да се глобална правила дефинишу на нивоу шеме, док се специјална правила могу дефинисати на нивоу појединачног ентитета или поља.

3.2.6 Везе ентитета

Односи између ентитета дефинишу се помоћу конструкције *relationship*. Полазни ентитет наводи се после кључне речи *from*, а циљни ентитет након кључне речи *to*.

```
relationship UserHasProfile {
  from User
  to Profile
  properties {
    linked_at: datetime {
      boundary
    }
  }
  config {
    strategy: one-to-one
    generate: 30
    include: [
      { linked_at: "2026-01-01T10:00:00" }
    ]
  }
}
```

У оквиру везе могу се дефинисати и својства помоћу блока *properties*. Својства се записују на исти начин као и поља ентитета. Имају назив, тип и опциона ограничења.

Конфигурација везе дефинише начин генерисања односа између инстанци ентитета. Подржане су стратегије *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many*. Додатно се могу дефинисати минимални и максимални степен повезаности помоћу опција *minDegree* и *maxDegree*.

У тренутној имплементацији, дефинисане везе се користе приликом генерисања *Cypher* скрипти за *Neo4j* графну базу података, док остали генератори обрађују структуре дефинисане ентитетима.

3.2.7 Експлицитно дефинисање тестних случајева

Поред аутоматског генерисања вредности, *TestDataGen* омогућава кориснику да експлицитно наведе тестне случајеве који морају бити укључени у резултат. Они се дефинишу помоћу опције *include*. Сваки тестни случај се представља као скуп додела вредности пољима.

```
relationship StudentEnrollment {
  from User
  to Course
  properties {
    progress_percentage: number {
      range 0..100,
      boundary
    }
    status: enum ["enrolled", "completed", "dropped"]
  }
  config {
    strategy: many-to-many
    generate: 100
    minDegree: 1
    maxDegree: 4
    include: [
      { progress_percentage: 100.0, status: "completed" },
      { progress_percentage: 0, status: "dropped" }
    ]
  }
}
```

Додела вредности има облик *name: value*. Вредност може да буде скаларна вредност, листа скаларних вредности или *null*. Пример листе скаларних вредности је дат у наставку:

```
tags: [ "admin", "user" ]
```

Овим механизмом кориснику је омогућено да, поред вредности које систем самостално генерише на основу дефинисаних стратегија, зада и конкретне случајеве који су од значаја за тестирање. То је посебно корисно за познате проблематичне случајеве, регресионо тестирање и проверу специјалних комбинација вредности.

3.2.8 Комплетан пример

Следећи пример показује повезивање већине описаних конструкција конкретне синтаксе у једној *TestDataGen* спецификацији:

```
schema ComprehensiveSystem {
  description: "Master rad"
  seed: 98765
  strategy: smart
  combination_strategy: pairwise

  entity User {
```



```

    fields {
      id: uuid
      email: email {
        unique,
        include [null, empty, invalid]
      }
      username: username {
        unique
      }
      age: number {
        range 18..99,
        boundary,
        partitions [27, 67]
      }
      status: enum ["active", "suspended", "pending"] {
        coverage 100
      }
    }
    config {
      generate: 50
      combination_strategy: each-used
      include: [
        { id: "00000000-0000-0000-0000-000000000001", email: "admin@system.local",
username: "superadmin", age: 30, status: "active" },
        { id: "00000000-0000-0000-0000-000000000002", email: null, username:
"guest_user", age: 17, status: "pending" }
      ]
    }
  }

  entity Profile {
    fields {
      id: uuid
      bio: string
      birth_date: date {
        range "1950-01-01".."2005-12-31",
        boundary
      }
      is_verified: boolean {
        coverage 100
      }
    }
    config {
      generate: 30
    }
  }

  entity Course {
    fields {
      id: uuid
      title: productName
      price: number {
        range 0..500.0,
        boundary,
        precision 2,
        special ["0", "99.99", "499.99"]
      }
    }
  }

```

```
    }
    capacity: number {
      range 10..200,
      partition 5
    }
  }
  config {
    generate: 20
    combination_strategy: full
  }
}

relationship UserHasProfile {
  from User
  to Profile
  properties {
    linked_at: datetime {
      boundary
    }
  }
  config {
    strategy: one-to-one
    generate: 30
    include: [
      { linked_at: "2026-01-01T10:00:00" }
    ]
  }
}

relationship StudentEnrollment {
  from User
  to Course
  properties {
    progress_percentage: number {
      range 0..100,
      boundary
    }
    status: enum ["enrolled", "completed", "dropped"]
  }
  config {
    strategy: many-to-many
    generate: 100
    minDegree: 1
    maxDegree: 4
    include: [
      { progress_percentage: 100.0, status: "completed" },
      { progress_percentage: 0, status: "dropped" }
    ]
  }
}
}
```

Наведени пример илуструје основну организацију *TestDataGen* спецификације. На нивоу шеме дефинисана су глобална подешавања. У оквиру ентитета су описани структура података, типови поља и ограничења. Конфигурацијом ентитета одређен је број инстанци које треба генерисати. Везом је описан однос између ентитета и начин генерисања.

3.3 Семантика

Семантика језика *TestDataGen* дефинише значење валидне спецификације, односно начин на који се информације описане спецификацијом користе за генерисање тестних података. Док конкретна синтакса одређује начин записивања спецификације, семантика одређује шта такав запис представља и какав резултат треба да произведе.

Након парсирања текстуалне спецификације, *textX* формира објектни модел на основу правила дефинисаних у граматичи. Добијени модел садржи објекте који одговарају елементима спецификације, као што су шема, ентитет, поље, ограничење, конфигурациони параметар и веза између ентитета. Над добијеним моделом врши се даља обрада. Она обухвата проверу семантичке исправности, одређивање правила генерисања, избор вредности и њихово комбиновање. Коначан резултат представља скуп генерисаних тестних података у изабраном излазном формату.

Семантичка правила језика *TestDataGen* одређују значење основних концепата језика. Шема представља целокупну спецификацију и садржи глобална подешавања која се могу користити као подразумеване вредности приликом генерисања. Ентитет представља структуру за коју се генеришу подаци, док његова поља одређују структуру и карактеристике генерисаних вредности. Веза описује однос између два ентитета.

Синтаксна исправност спецификације не гарантује да спецификација има смислено значење. Због тога се након парсирања врши семантичка анализа модела. Тиме се проверавају услови који се не могу или их није погодно дефинисати самом граматиком.

Један од примера представља референце између ентитета. Када се поље дефинише као референца, референтни ентитет мора постојати у оквиру шеме. На пример:

```
entity Order {
    fields {
        user: ref User
    }
}
```

У наведеном примеру *User* мора бити дефинисан као ентитет у шеми. Иако се постојање таквог ентитета може проверити пре парсирања, његова исправност представља семантичко, а не синтаксно правило.

Слично томе, вредности и ограничења поља морају бити међусобно усаглашени. На пример, ограничење *precision* има смисла за нумеричке вредности, док се за поља типа *boolean* не може применити исто ограничење. Семантичка анализа треба да обезбеди да су конфигурација и ограничења применљиви на тип поља над којим су дефинисани.

Семантичка правила се примењују и на конфигурацију генератора. Број података који се генерише мора представљати позитивну вредност. Изабране стратегије морају бити подржане за дати тип података и конфигурацију. На исти начин, минимални и максимални степен повезаности код веза морају бити међусобно усаглашени.

3.3.1 Семантика стратегија генерисања

Један од основних делова семантике језика *TestDataGen* представљају стратегије генерисања. Стратегије одређују начин избора вредности за одређено поље. Подржане су стратегије *random*, *boundary*, *partition* и *smart* [5].

Стратегија *random* генерише вредности случајним избором из скупа вредности које одговарају типу и дефинисаним ограничењима. Стратегија *boundary* користи граничне вредности домена, као и вредности непосредно изнад и испод дефинисаних граница када је то применљиво. Стратегија *partition* користи представнике дефинисаних партиција еквиваленције. Стратегија *smart* комбинује расположиве стратегије како би се добио скуп тестних вредности који обезбеђује бољу покривеност дефинисаних случајева.

Стратегија може бити дефинисана на нивоу шеме, конфигурације ентитета или појединачног поља. На тај начин је омогућено дефинисање подразумеваног понашања, уз могућност да се оно измени на нижем нивоу. Уколико стратегија није експлицитно наведена за поље, користи се одговарајућа подразумевана стратегија.

3.3.2 Семантика ограничења

Ограничења над пољима представљају правила која одређују које вредности могу бити генерисане и које тестне случајеве треба укључити у резултат. Ограничења и њихове дефиниције су приказани у табели 1.

Табела 1: Табела ограничења

Ограничења	Шта дефинише
<i>range</i>	Дозвољени опсег вредности
<i>boundary</i>	Приликом генерисања се морају размотрити гранични случајеви
<i>precision</i>	Број децималних места на које се заокружују генерисане нумеричке вредности
<i>partition</i>	Број партиција на које се домен дели
<i>partitions</i>	Конкретне вредности представника партиција
<i>unique</i>	Генерисане вредности поља морају бити јединствене
<i>include</i>	Експлицитно укључивање вредности као што су <i>null</i> , празна или невалидна вредност
<i>special</i>	Кориснички наведене вредности које треба укључити у генерисани скуп вредности
<i>coverage</i>	Захтевани ниво покривености

Ограничења *partition* и *partitions* представљају два различита начина дефинисања партиционисања. Ограничење *partition* одређује број партиција које генератор треба да формира, а *partitions* омогућава кориснику да наведе експлицитне представнике партиција. Није могуће користити их заједно. Такође, користе се само код типова *number*, *date* и *datetime*.

Ограничење *precision* одређује број децималних места на које се заокружују генерисане вредности типа *number*. Вредност мора да буде ненегативан цео број. Такође, ова вредност се користи и код одабира граничних вредности (*boundary*).

Ограничење *include* омогућава експлицитно укључивање вредности које можда не би биле изабране случајним генерисањем. Могуће је укључити само *null*, празну или невалидну вредност. Разлика код ограничења *special* је у томе што је могуће у генерисани скуп укључити експлицитно наведене вредности. Вредности се прослеђују као текстуални литерали. Води се рачуна о томе да се наведе барем једна вредност за укључивање.

Ограничење *coverage* одређује захтевани ниво покривености. Задата вредност може бити од 0 до 100.

Више ограничења може бити примењено над истим пољем. У том случају, генератор приликом избора вредности мора да узме у обзир сва дефинисана ограничења. На пример, уколико корисник жели да се приликом генерисања узму у обзир дефинисани опсег, граничне вредности и представници партиција домена, запис би био следећи:

```
age: number {
  range 18..99,
  boundary,
  partitions [27, 67]
}
```

3.3.3 Семантика комбиновања вредности

Након што се за појединачна поља одреде могуће вредности, потребно је одредити на који начин ће се те вредности комбиновати у тестне случајеве. Језик *TestDataGen* подржава три стратегије комбиновања: *full*, *pairwise* и *each-used*.

Стратегија *full* формира све могуће комбинације вредности, односно Декартов производ скупова вредности појединачних поља. Оваквим приступом обезбеђује се потпуна покривеност комбинација, али број случајева може значајно порастати са бројем поља и њихових могућих вредности.

Стратегија *pairwise* настоји да обезбеди да се сваки пар вредности из различитих поља појави најмање једном, уколико је то могуће у оквиру генерисаних случајева. На тај начин, број тестних случајева се смањује у односу на потпуно комбиновање, уз задржавање покривености парова вредности.

Стратегија *each-used* обезбеђује да се свака генерисана вредност употреби најмање једном. Ова стратегија додатно смањује број комбинација када није неопходно тестирати све парове или све комбинације вредности.

3.3.4 Семантика референци

Референтна поља омогућавају повезивање генерисаних инстанци различитих ентитета. За поље типа *ref* генератор мора да обезбеди да изабрана вредност представља референцу на постојећу инстанцу одговарајућег ентитета.

Уколико је референца дефинисана као колекција, само тад је могуће ограничити њену величину минималним и максималним бројем елемената. Генератор ће проверавати да ли су вредности ненегативни бројеви и да ли је минимална вредност заиста мања од максималне. Ограничење *boundary* је дозвољено код референци само ако је дефинисано претходно ограничење *count*. Ограничење *count* можемо видети и на примеру где генерисана инстанца може садржати између једне и пет референци на инстанце ентитета *User*:

```
users: ref User[] count 1..5
```

3.3.5 Семантика веза

Везе између ентитета додатно дефинишу начин повезивања њихових инстанци. Стратегије *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many* одређују кардиналост односа. Опције *minDegree* и *maxDegree* дефинишу дозвољени број веза једне инстанце са инстанцама другог ентитета. За стратегију *one-to-one* није дозвољено дефинисање ограничења броја веза, јер је он дефинисан самом кардиналношћу стратегије.

Горња граница скупа свих генерисаних веза је описана ограничењем *generate*, али број генерисаних веза може бити мањи у зависности од изабране стратегије.

Својства веза не могу да буду типа референца на ентитет.

3.3.6 Семантика експлицитно дефинисаних тестних случајева

Експлицитно дефинисани случајеви представљају вредности које корисник захтева да буду присутне у резултату генерисања. Оне се обрађују заједно са аутоматски генерисаним вредностима. Неопходно је да не постоји више додела истом пољу при навођењу једног тестног случаја, такође да су сва поља постојећа за тај ентитет и да су сви типови вредности одговарајући.

На пример, корисник захтева да резултујући скуп података садржи наведене комбинације вредности:

```
entity User {
  fields {
    id: uuid
    email: email {
      unique,
      include [null, empty, invalid]
    }
  }
}
```

```
    username: username {
      unique
    }
    age: number {
      range 18..99,
      boundary,
      partitions [27, 67]
    }
    status: enum ["active", "suspended", "pending"] {
      coverage 100
    }
  }
  config {
    generate: 50
    combination_strategy: each-used
    include: [
      { id: "00000000-0000-0000-0000-000000000001", email: "admin@system.local",
username: "superadmin", age: 30, status: "active" },
      { id: "00000000-0000-0000-0000-000000000002", email: null, username:
"guest_user", age: 17, status: "pending" }
    ]
  }
}
```

На овај начин корисник може експлицитно да дефинише познате граничне, неважеће или друге специфичне случајеве који су од значаја за тестирање.

3.3.7 Процес обраде спецификације

Обрада *TestDataGen* спецификације може се представити као низ корака. Текстуална спецификација се парсира на основу граматике дефинисане у *textX*-у. Резултат парсирања је објектни модел спецификације. Након тога се над моделом врши семантичка анализа и процес провере дефинисаних правила.

Када је модел семантички исправан, на основу његове конфигурације бирају се одговарајуће стратегије генерисања. За свако поље одређује се скуп могућих вредности, при чему се узимају у обзир његов тип, ограничења и изабрана стратегија. Добијене вредности се затим комбинују према изабраној стратегији комбиновања.

Уколико спецификација садржи експлицитно дефинисане тестне случајеве, они се укључују у резултујући скуп. Након генерисања и комбиновања вредности, добијени модел тестних података прослеђује се одговарајућем генератору излазног формата.

На тај начин семантика језика *TestDataGen* повезује структуру описану апстрактном и конкретном синтаксом са стварним понашањем система. Корисник специфицира шта жели да тестира, док семантичка правила језика одређују како ће се та спецификација интерпретирати и како ће се на основу ње добити тестни подаци.

Архитектура језика *TestDataGen*

Архитектура система *TestDataGen* организована је тако да раздвоји опис тестних података, њихову обраду и генерисање резултујућих формата. Основу система представља текстуална спецификација написана у језику *TestDataGen*, која се помоћу граматике дефинисане у алату *textX* парсира у објектни модел спецификације. Над добијеним моделом се врши семантичка провера и генерисање тестних података, након чега се резултат прослеђује одговарајућем генератору излазних фајлова.

Оваква организација омогућава да различити генератори користе исти модел и исти поступак генерисања података. Начин представљања података зависи само од изабраног излазног формата. Подржани су генератори за: *SQL*, *JSON*, *CSV*, *YAML*, *Neo4j* (то јест *Cypher* скрипту за графну базу) и *HTML* извештај о покривености. Само *Neo4j* обрађује везе између ентитета, док сви остали генератори обрађују само податке дефинисане ентитетима.

Целокупна структура пројекта организована је у више модула. Граматика језика се налази у оквиру модула за граматiku, генератори у фолдеру *generators*, док су стратегије генерисања и комбиновања имплементирани у оквиру директоријума *strategies*. Поред ових компоненти, систем садржи модул за учитавање модела и граматике, командну линију за покретање генерације података, компоненте за интеграцију са алатом *Faker* за генерисање вредности различитих типова [6].

4.1 Основни ток обраде

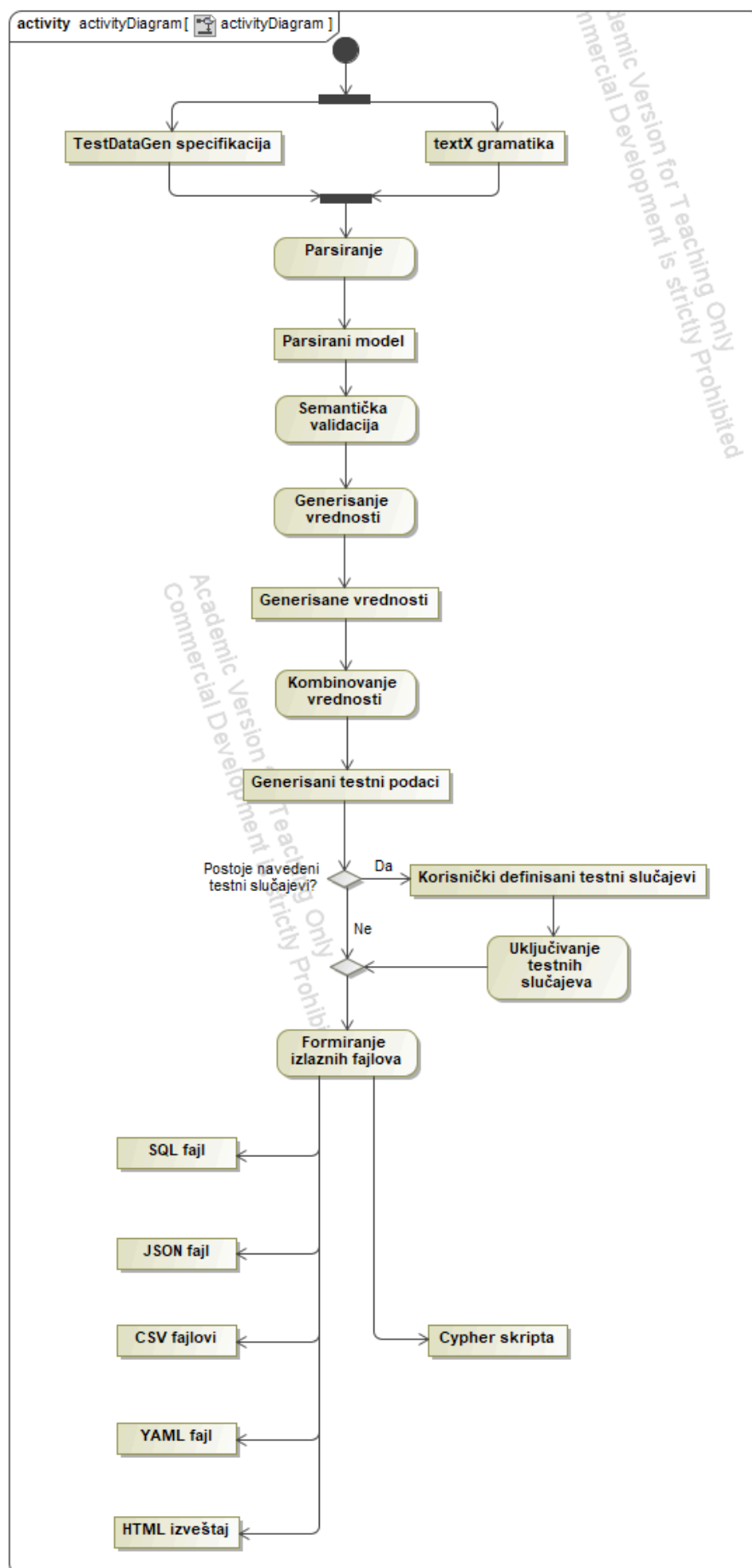
Обрада спецификације у систему *TestDataGen* може се посматрати као низ међусобно повезаних корака. На почетку се учитава текстуална спецификација из улазног фајла. Модул за учитавање граматике учита *textX* метамодел и на основу њега парсира спецификацију, чиме се добија модел који садржи структуру шеме, ентитета, поља, ограничења, конфигурације и везе између ентитета ако су дефинисане.

Након успешног парсирања, над моделом се примењују правила семантичке валидације. Њихова улога је да провере исправност спецификације са становишта правила језика која се не могу у потпуности проверити граматиком. На тај начин се пре генерисања спречава обрада неисправно дефинисаних модела.

Након валидације, модел се прослеђује поступку генерисања. На основу конфигурације шеме, ентитета и поља се одређује стратегија генерисања и број података који треба произвести. Стратегије генерисања су имплементирани као засебне компоненте, што омогућава њихово коришћење независно од конкретног излазног формата. У систему су подржане стратегије *random*, *boundary*, *partition* и *smart* [5]. Након избора вредности за појединачна поља, њихово комбиновање у тестне случајеве врши се применом једне од стратегија *full*, *pairwise* или *each-used*. Оваква организација је у складу са поделом стратегија у засебан модул система.

Добијене вредности се затим комбинују у тестне случајеве према изабраној стратегији комбиновања. Уколико спецификација садржи експлицитно дефинисане тестне случајеве, они се такође укључују у резултујући скуп тестних случајева. Након завршетка овог поступка, генерисани подаци се прослеђују једном или више излазних генератора, у зависности од корисничког избора.

Основни ток може се приказати као на слици 3:

Слика 3: Дијаграм активности тока генерисања тестних података у систему *TestDataGen*.

4.2 Интерфејс командне линије

Корисник системом управља преко интерфејса командне линије (енг. *Command Line Interface*, *CLI*). *CLI* је имплементиран коришћењем библиотеке *Click*. Он представља улазну тачку за покретање генерисања.

Команда *generate* прима назив спецификације и омогућава задавање директоријума излаза, једног или више формата, почетне вредности генератора случајних бројева и опције за писање иако датотека већ постоји. Подржани формати су: *sql*, *json*, *report*, *csv*, *yaml* и *neo4j*.

Могуће је једним позивом команде тражити генерисање више различитих формата. Пример је приказан у наставку:

```
testdatagen generate master.tdata \
  --output ./output \
  --format sql,json,report,yaml,csv,neo4j
  --overwrite
```

CLI пре самог генерисања проверава да ли су тражени формати подржани, након чега се за сваки изабрани формат позива одговарајући генератор. На овај начин корисник не мора да покреће посебну команду за сваки формат.

Поред команде за генерисање, систем подржава засебну команду за проверу исправности спецификације без генерисања излазних података. Она је приказана у наредним примеру:

```
testdatagen validate master.tdata
```

4.3 Парсирање и модел језика

Први корак у обради спецификације представља њено парсирање. Граматика *TestDataGen*-а дефинише конструкције као што су шема, ентитет, поље, ограничења, конфигурације и везе.

За креирање метамодела се користи библиотека *textX* и њена метода *metamodel_from_file* [4]. Приликом креирања метамодела региструју се процесори модела и правила за валидацију, након чега се добијени метамодел поново користи приликом учитавања спецификације.

Резултат парсирања није текстуални приказ спецификације, него структурни модел који садржи елементе спецификације. Генератори затим приступају овом моделу и његовим ентитетима, пољима, конфигурацијама и везама.

Оваква организације омогућава да се једном изграђени модел користи у више различитих делова система. *SQL*, *JSON*, *CSV*, *YAML* и генератор *HTML* извештај користе структуре ентитета, док *Neo4j* генератор поред ентитета обрађује и дефинисане везе.

4.4 Семантичка валидација

После парсирања врши се семантичка провера модела. Њена сврха је да се открију спецификације које су синтаксно исправне, али немају исправно или смислено значење.

Валидација је издвојена у посебан модул. У имплементацији постоје посебни типови грешака. Они се користе за различите врсте неправилности међу којима су грешке у опсегу, набројивим вредностима, броју генерисаних података, прецизности, конфигурацији везе и вредностима експлицитно дефинисаних тестних случајева.

Овај слој има важну улогу јер омогућава да се провере ограничења која зависе од контекста. На пример, ограничење *precision* има смисла за нумеричке вредности, док исти концепт није применљив за логичке вредности. Слично томе, вредност минималног и максималног степена код веза морају бити међусобно усаглашене (минимални не може бити већи од максималног).

Раздвајање парсирања и семантичке валидације омогућава да граматика остане фокусирана на структуру језика, док се правила зависна од значења обрађују у семантичком слоју.

4.5 Генерисање тестних вредности

Након успешне валидације започиње генерисање тестних вредности. Овај део система представља централни механизам *TestDataGen*-а. Ту се примењују стратегије за паметни избор вредности [5].

За свако поље систем анализира његов тип, ограничења и изабрану стратегију. У зависности од спецификације могу се користити случајно генерисање (енг. *random*), анализа граничних случајева (енг. *boundary*), партиционисање на класе еквиваленције (енг. *partition*) или комбинација ових приступа кроз паметну стратегију (енг. *smart*).

Код за обраду поља формира скуп захтеваних вредности. Приликом примене стратегије граничних случајева (енг. *boundary*) генеришу се вредности повезане са границама опсега. Код стратегије партиционисања на класе еквиваленције (енг. *partition*) генеришу се представници партиција. За нумеричке, датумске и временске вредности (енг. *datetime*), ове стратегије се ослањају на дефинисани опсег.

Посебно се обрађују набројиви типови (енг. *enum*). Код њих паметна стратегија може да обезбеди покривеност свих дефинисаних вредности.

Поред тога, обрађују се специјалне вредности и случајеви дефинисани ограничењима *special* и *include*.

Овим се постиже да се стратегије генерисања вредности не налазе у самом генератору излазног формата. Њихов резултат представљају вредности које касније могу бити серијализоване у различите формате.

4.6 Комбиновање вредности

Након што се одреде вредности за појединачна поља, оне се комбинују у тестне случајеве. *TestDataGen* подржава три начина комбиновања: *full*, *each-used* и *pairwise*.

Код стратегије *full* генеришу се све комбинације вредности поља. Код стратегије *each-used* обезбеђује се да ће се свака генерисана вредност појавити најмање једном. Стратегија *pairwise* настоји да се сваки пар вредности из различитих поља обухвати најмање једним тестних случајем.

Овај део обраде је независан од конкретног излазног формата. То значи да без обзира на то који излазни формат корисник жели, комбинације вредности се формирају на истом месту у систему.

У имплементацији генератора, овај процес је организован као низ заједничких корака (енг. *pipeline*):

- прикупљање вредности
- комбиновање у редове попуњене до захтеваног броја записа
- додавање експлицитно задатих случајева
- решавање референци.

Код *SQL* генератора је овај редослед експлицитно описан у самом моделу.

4.7 Обрада референци

Посебан део обраде представљају референце између ентитета. Када једно поље представља референцу (енг. *ref*) на други ентитет, његова вредности мора да буде повезана са стварном инстанцом референцираног ентитета.

Због тога систем током генерисања памти идентификаторе већ дефинисаних ентитета. За сваки ентитет одређује се поље које се користи као идентификатор, при чему се прво тражи поље под називом *id*, затим поље типа *uuid*, а као последња могућност користи се прво поље ентитета.

Након генерисања вредности, референтна поља се разрешавају тако што се специјална интерна вредност *ref* замењује идентификатором одговарајуће генерисане инстанце.

Овај механизам омогућава да генератори, који производе податке за структуре са референцама, задрже конзистентност између повезаних ентитета.

4.8 Генератори излазних формата

Након завршетка заједничког процеса генерисања, резултат се прослеђује изабраном генератору излазног формата. У пројекту су генератори издвојени у посебан фолдер, *generators*. Постоји шест генератора излазних резултата, а то су: *SQL*, *JSON*, *CSV*, *YAML*, *Neo4j* (производи *Cypher* скрипту за графну базу) и *HTML* извештај о покривености.

Основна предност овакве организације јесте раздвајање логике избора и комбиновања вредности од логике форматирања конкретног излазног формата. На пример, *CSV* генератор не мора да зна на који начин се одређују граничне вредности. Корази *pipeline*-а који претходе серијализацији обезбеђују да су вредности већ генерисане и комбиноване у одговарајуће тестне случајеве.

Генератор затим прилагођава добијене податке захтевима конкретног формата. Тако *SQL* генератор производи *INSERT* исказе, *JSON* генератор структуриране *JSON* податке, *CSV* генератор производи *CSV* датотеке, *YAML* генератор *YAML* представу, док *Neo4j* генератор производи *Cypher* скрипту.

Овакво раздвајање омогућава да додавање новог излазног формата не захтева измену логике парсирања, семантичке анализе или стратегија генерисања.

4.8.1 Neo4j генератор

Neo4j генератор представља изузетак у односу на остале генераторе јер не обрађује само структуре ентитета већ и везе између њих.

Генератор прво обрађује податке који представљају чворове графа. За сваки чвор користи назив ентитета као ознаку, а својства ентитета претвара у својства *Neo4j* чвора. За сваки генерисани чвор формира се *CREATE* исказ. Након тога обрађују се везе. За сваку везу генератор проналази одговарајуће полазне и циљне чворове на основу њених идентификатора и формира *Cypher CREATE* и *MATCH* исказе.

На тај начин архитектура омогућава да се специфичности графовског модела обрађују само у генератору који их захтева. Остали генератори остају фокусирани само на ентитетске податке.

4.8.2 Генерисање HTML извештаја о покривености

Поред генератора тестних података, систем садржи и генератор извештаја о покривености. Његова улога је да прикаже у којој мери су вредности које су захтеване изабраним стратегијама заиста присутне у генерисаном скупу.

Механизам покривености формира скуп захтеваних вредности на основу активних стратегија и ограничења. У тај скуп улазе, између осталог, граничне вредности, представници партиција и вредности специјалних случајева. Покривеност се израчунава на основу односа броја захтеваних вредности које су заиста присутне у генерисаном скупу и укупног броја захтеваних вредности.

Ово омогућава да извештај не приказује само генерисане податке, већ и информације о томе да ли су испуњени захтеви постављени стратегијама генерисања.

4.9 Коришћене технологије

Имплементација језика *TestDataGen* заснована је пре свега на софтверском језику *Python*. За дефинисање и парсирање ЈСД-а користи се *textX*. Да би систем обезбедио реалистичне вредности користи се библиотека *Faker*. За генерисање излазних докумената заснованих на шаблонима користи се *Jinja2*. Ове технологије су наведене и у документацији пројекта.

За интеграцију са корисничким интерфејсом користи се *Click*, док су различити генератори издвојени у посебан директоријум. Оваква модуларност организације омогућава независан развој и тестирање појединачних делова система.

4.10 Битне карактеристике архитектуре

Описана архитектура обезбеђује неколико важних особина система.

Прва је раздвајање одговорности:

- граматика је одговорна за синтаксу
- валидације за семантичку исправност
- стратегије за избор вредности
- механизам комбиновања за формирање тестних случајева
- генератори за представљање тестних случајева у конкретном формату.

Друга је поновна употребљивост заједничке логике. Сви генератори, са изузетком специфичне обраде коју захтева *Neo4j*, користе исти концептуални модел и заједнички механизам генерисања. На тај начин се избегава дуплирање алгоритама у различитим генераторима.

Трећа је проширивост. Нови излазни формат може се додати као засебан генератор без промене граматике и основних стратегија генерисања. Слично томе, нова стратегија генерисања може се имплементирати у делу система задуженом за избор вредности без потребе да се мењају постојећи генератори.

Коначно, архитектура подржава истовремено генерисање више формата. *CLI* прихвата листу формата, а за сваки изабрани формат позива одговарајући генератор. На тај начин једна *TestDataGen* спецификација представља јединствени извор података из ког се могу добити различите представе истог скупа тестних података.

Глава 5

Разлози избора излазних формата

TestDataGen омогућава генерисање тестних података у више различитих формата. На тај начин се генерисани подаци могу користити у различитим окружењима и за различите потребе тестирања. Избор више формата омогућава да иста спецификација буде извор тестних података намењених различитим типовима система, база података и алата. Поред формата у којима се генеришу тестни подаци, систем подржава и генерисање *HTML* извештаја о покривености.

5.1 SQL формат

SQL формат је погодан директно за коришћење генерисаних тестних података у релационим базама података. *TestDataGen* генерише *SQL* скрипту која садржи исказе за унос података, чиме се омогућава једноставно попуњавање табела тестним подацима.

Овај формат је посебно користан приликом тестирања апликација које користе релационе базе података. Генерисани подаци могу се искористити за тестирање упита, пословне логике и рада апликације над већом количином података, без потребе за њиховим ручним уносом.

У наставку можемо видети пример генеризаних података у *SQL* формату. У заглављу се види да је конкретно генерисано 50 тестних случајева на основу структуре ентитета *User*. Називи колона (*id*, *email*, *username*, *age*, *status*) одговарају пољима ентитета, а свака додата ставка представља једну генерисану инстанцу.

```
-- User (50 records)
INSERT INTO users (id, email, username, age, status) VALUES
('00000000-0000-0000-0000-000000000001', 'admin@system.local', 'superadmin', 30.0, 'active'),
('00000000-0000-0000-0000-000000000002', 'kellyriggs@example.net', 'guest_user', 17.0,
'pending'),
('f86c882c-f9af-4a56-8215-c2b76947c104', 'cody82@example.com', 'adamsdaniel', 98.0,
'active'),
```

5.2 JSON формат

JSON је погодан за размену структурираних података између различитих апликација и сервиса. Због своје структуре, једноставности и широке подршке у програмским језицима, често се користи приликом комуникације клијентских и серверских компоненти.

У *TestDataGen* систему *JSON* формат омогућава да се генерисани тестни подаци користе за тестирање *REST API*-ја и других система који прихватају или производе *JSON* податке. То омогућава да једна спецификација буде искоришћена за припрему улазних података за аутоматизоване тестове сервиса.

У наставку можемо видети исечак генеризаних података у *JSON* формату. Исечак се односи само на генерисане податке на основу дефинисаних ентитета, за конкретан случај ентитета *User*. Приказано је да се за свако поље наводи назив поља и генерисана вредност.

```
"entities": {
  "User": [
    {
      "id": "00000000-0000-0000-0000-000000000001",
      "email": "admin@system.local",
      "username": "superadmin",
      "age": 30.0,
```

```
    "status": "active"
  },
  {
    "id": "00000000-0000-0000-0000-000000000002",
    "email": "kellyriggs@example.net",
    "username": "guest_user",
    "age": 17.0,
    "status": "pending"
  },
  {
    "username": "adamsdaniel",
    "age": 98.0,
    "status": "active",
    "id": "f86c882c-f9af-4a56-8215-c2b76947c104",
    "email": "cody82@example.com"
  },
]
}
```

5.3 HTML извештај о покривености

Поред генерисања тестних података, *TestDataGen* омогућава генерисање *HTML* извештаја о покривености. Његова сврха није представљање података за даљи увоз у други систем, већ анализа квалитета скупа генерисаних тестних података.

Извештај приказује у којој мери су вредности које су захтеване изабраним стратегијама и ограничењима заиста присутне у генерисаном скупу. На тај начин корисник може да провери да ли су, на пример, обухваћене граничне вредности, представници партиција и специјалне вредности. Ово је посебно значајно јер омогућава да се поред самих генерисаних података добије и информација о њиховом квалитету.

Пример *HTML* извештај о покривености можемо видети на слици [4](#):

TestDataGen — Coverage Report

Schema: ComprehensiveSystem Strategy: smart Seed: 98765 Generated: 2026-09-02 22:04:29

75.0% 30 / 40 required cases covered across 3 entities

Course 63.2% 20 records | 12/19 cases

FIELD	TYPE	STRATEGIES	REQUIRED	COVERED	COVERAGE
id	uuid	random	0	0	n/a
title	productName	random	0	0	n/a
price	number	BVA EP special	10	3	30.0% Missing: 0 250.0 416.67 499.99 500.0 83.33 99.99
capacity	number	BVA EP	9	9	100.0%

Profile 100.0% 30 records | 9/9 cases

FIELD	TYPE	STRATEGIES	REQUIRED	COVERED	COVERAGE
id	uuid	random	0	0	n/a
bio	string	random	0	0	n/a
birth_date	date	BVA EP	7	7	100.0%
is_verified	boolean	bool-coverage	2	2	100.0%

User 75.0% 50 records | 9/12 cases

FIELD	TYPE	STRATEGIES	REQUIRED	COVERED	COVERAGE
id	uuid	random	0	0	n/a
email	email	edge	3	0	0.0% Missing: None invalid-email-format
username	username	random	0	0	n/a
age	number	BVA EP	6	6	100.0%
status	enum	EP	3	3	100.0%

Relationships

NAME	FROM	TO	STRATEGY	GENERATE	INCLUDE CASES	PROPERTIES
UserHasProfile	User	Profile	one-to-one	30	1	linked_at (datetime)
StudentEnrollment	User	Course	many-to-many	100	2	- progress_percentage (number) - status (enum)

Слика 4: Пример HTML извештаја генерисаног користећи TestDataGen.

5.4 CSV формат

CSV формат представља једноставан начин представљања табеларних података. Њихова предност је у широкој употреби различитих алата, база података и програмских језика, као и у једноставној структури која омогућава лаку обраду већих количина података.

У оквиру TestDataGen-а CSV је погодан за генерисање тестних скупова података који се могу даље обрађивати у алатима за анализу података, увозити у базу података или користити као улаз у аутоматизованим тестовима. Посебно је погодан када је потребан већи број записа представљених у облику редова и колона.

Неопходно је нагласити да се за потребе овог формата генерише фолдер за шему, за сваки ентитет посебна CSV датотека и текстуалну датотеку са информацијама о генерисању. Ту се види велика разлика у односу на остале формате где је излаз само једна датотека.

У наставку можемо видети пример генеризаних података у CSV формату. Заглавље одговара пољима ентитета *User*, а сваки ред једној инстанци.

```
id,email,username,age,status
00000000-0000-0000-0000-000000000001,admin@system.local,superadmin,30.0,active
00000000-0000-0000-0000-000000000002,kellyriggs@example.net,guest_user,17.0,pending
f86c882c-f9af-4a56-8215-c2b76947c104,cody82@example.com,adamsdaniel,98.0,active
```

5.5 YAML формат

YAML је текстуални формат намењен представљању структурираних података, при чему је нагласак стављен на читљивост података и конфигурације за човека. Подржава хијерархијску структуру и погодан је за представљање сложених података.

Генерисање YAML-а у *TestDataGen*-у омогућава употребу тестних података у системима и алатима који користе овај формат. Такође, једноставно је прегледање и ручна измена генерисаних података. Због тога се може користити за припрему тестних конфигурација и структурираних тестних сценарија.

У наставку можемо видети пример генеризаних података у YAML формату. Исечак се односи само на генерисане податке на основу дефинисаних ентитета, за ентитет *User* приказано је да свака генерисана инстанца садржи вредност поља у YAML структури.

```
entities:
  User:
    - id: 00000000-0000-0000-0000-000000000001
      email: admin@system.local
      username: superadmin
      age: 30.0
      status: active
    - id: 00000000-0000-0000-0000-000000000002
      email: kellyriggs@example.net
      username: guest_user
      age: 17.0
      status: pending
    - username: adamsdaniel
      age: 98.0
      status: active
      id: f86c882c-f9af-4a56-8215-c2b76947c104
      email: cody82@example.com
```

5.6 Neo4j формат

За разлику од претходно наведених формата, *Neo4j* излаз је намењен раду са графном базом података. *TestDataGen* генерише *Cypher* скрипту која омогућава креирање чворова и веза између њих у графној бази података *Neo4j*.

Овај излаз је значајан јер *TestDataGen* подржава моделовање односа између ентитета помоћу конструкције *relationship*. На тај начин се, поред самих података ентитета, могу генерисати и њихове везе. Ово омогућава припрему тестних графова и тестирање апликација чији подаци имају изражену структуру односа.

У наставку можемо видети пример генеризаних података у *Neo4j* формату. Конкретно је генерисано 100 чворова према структурама ентитета и 130 веза између ентитета.

```
// --- Node Definitions (records 100) ---
CREATE (:User {id: "00000000-0000-0000-0000-000000000001", email: "admin@system.local",
username: "superadmin", age: 30.0, status: "active"});

CREATE (:Profile {id: "3e488f08-c673-41bc-8362-59c67f999b44", bio: "Dinner check writer into.
Focus spring really so. Series western simple hard director in.", birth_date: "1950-01-02",
```



```
is_verified: false});
```

```
// --- Relationship Definitions (records 130) ---
```

```
MATCH (from:User {id: "00000000-0000-0000-0000-000000000001"}), (to:Profile {id: "3e488f08-c673-41bc-8362-59c67f999b44"})
```

```
CREATE (from)-[:USERHASPROFILE {linked_at: "2026-01-01T10:00:00"}]->(to);
```

5.7 Значај подршке више формата

Подршка различитих излазних формата омогућава да се једна *TestDataGen* спецификација користи у различитим контекстима без промене описа тестних података. Корисник једном дефинише структуру, ограничења и стратегије генерисања, а затим може да изабере формат који одговара систему или алату у којем ће се тестни подаци користити.

Опис и начин генерисања тестних података су раздвојени од њихове конкретне представе. *SQL* и *Neo4j* су погодни за рад са базама података. *JSON* и *YAML* за системе који користе структуриране текстуалне податке, а *CSV* за табеларне скупове података. *HTML* извештај, са друге стране, пружа увид у покривеност тестних случајева.

Организација *TestDataGen*-а, описана у претходном поглављу, представља једну од предности система. Исти опис тестних података може бити употребљен за различите потребе тестирања, без поновног дефинисања самих података и правила њиховог генерисања.

Глава 6

Имплементација додатних генератора излазних формата

У оквиру овог мастер рада, имплементирана су три генератора излазних формата за систем *TestDataGen*. То су: *CSV*, *YAML* и *Neo4j* генератор. Циљ њихове имплементације је био проширивање постојећег система могућношћу представљања генерисаних тестних података у форматима који се разликују по начину организације и намени.

Имплементирани генератори интегрисани су у постојећи механизам система, тако да користе исти модел добијен парсирањем *TestDataGen* спецификације и исти поступак генерисања тестних података. На тај начин логика избора и комбиновања тестних података остаје одвојена од логике њиховог записивања у конкретан излазни формат. Постојећи механизам командне линије омогућава избор једног или више подржаних формата приликом једног покретања система. У документацији пројекта наведени су сви излазни формати: *SQL*, *JSON*, *HTML* извештај о покривености, *CSV*, *YAML* и *Neo4j*.

Сваки од имплементираних генератора има специфичан начин представљања резултата. *CSV* генератор организује податке у табеларном облику и креира посебну датотеку за сваки ентитет. *YAML* генератор представља генерисане ентитете у хијерархијској текстуалној структури. За потребе *Neo4j* генератора, у оквиру проширења система подржано је моделовање веза између ентитета. Концепт *relationship* моделује везу између ентитета, док *Neo4j* генератор те *relationship*-ове користи за формирање веза у графу. Тако да је кључна разлика *Neo4j* генератора, у односу на све остале генераторе, то што обрађује и податке о везама и њиховим својствима.

У наставку су детаљно описане имплементација и специфичности ова три генератора, са посебним освртом на начин на који преузимају резултат заједничког процеса генерисања, обрађују податке и формирају коначни излаз.

6.1 CSV генератор

CSV генератор представља један од три додатна генератора имплементирана у оквиру овог мастер рада. Његова улога је да резултат заједничког процеса генерисања тестних података прилагоди табеларном *CSV* формату [7]. При томе се не имплементира посебна логика за избор и комбиновање тестних вредности, већ се користе постојеће компоненте система. У самом генератору реализован је поступак припреме вредности специфичан за *CSV* формат и његово уписивање у излазне датотеке.

CSV генератор је имплементиран у оквиру модула *csv_generator*. Основна класа генератора је *CSVGenerator*, која као улаз прима парсирани *textX* модел спецификације. Приликом иницијализације преузима се *seed* из спецификације, уколико није експлицитно прослеђен, и иницијализује се *FakerTypeMapper* који се користи за генерисање вредности [6].

Посебна карактеристика имплементације је поновна употреба заједничког механизма за генерисање података. *CSV* генератор поново користи заједничке функције за прикупљање вредности, њихово комбиновање, обраду експлицитно задатих случајева и разрешавање референци, које су имплементиране у *sql_generator* модулу. На тај начин се исти поступак генерисања користи независно од тога да ли ће резултат бити записан у *CSV* или неки други излазни формат.

6.1.1 Формирање вредности

Пре уписивања у CSV датотеке, генерисане вредности потребно је прилагодити начину представљања података у CSV формату. У ту сврху имплементирана је функција *format_value_csv*, која обрађује различите типове вредности.

Функција посебно обрађује *None*, логичке вредности, бројеве, датуме, датум и време, текстуалне вредности и листе. *None* се представља празним пољем. Логичке вредности се записују малим словима (*true* и *false*). Датум, као и датум и време, се конвертују у ISO формат. Листе се представљају као вредности раздвојене тачком и зарезом унутар једне CSV ћелије. На тај начин се и листе идентификатора (настале код референци на више инстанци другог ентитета) представљају као једна вредност у CSV ћелији, при чему су појединачни идентификатори ентитета раздвојени тачком и зарезом.

Кључни део имплементације приказан је у листингу 1.

```
def format_value_csv(value: Any) -> Any:
    if value is None:
        return ""
    if isinstance(value, bool):
        return "true" if value else "false"
    if isinstance(value, (int, float)):
        return value
    if isinstance(value, datetime):
        return value.isoformat()
    if isinstance(value, date):
        return value.isoformat()
    if isinstance(value, list):
        return ";".join(" if v is None else str(format_value_csv(v)) for v in value)
    if isinstance(value, str):
        return value
    return str(value)
```

Листинг 1: Функција за формирање вредности за CSV формат

Ова функција представља један од специфичних делова CSV генератора, јер се њоме вредности добијене из заједничког механизма генерисања преводе у представу погодну за упис у CSV датотеку.

6.1.2 Генерисање CSV датотека

Резултат генерисања организован је тако да се за сваки ентитет креира посебна CSV датотека. Пре самог уписа, метода *generate* позива заједнички поступак *build*, након чега се добијени подаци прослеђују функцији за писање метаподатака и функцији за писање CSV датотека.

Упис података за појединачни ентитет реализован је методом *write_entity_csv*. Назив ентитета се користи за формирање назива датотеке. На пример за ентитет *User* креира се датотека *User.csv*. За упис се користи *csv.DictWriter*, при чему се називи колона преузимају из кључева првог генерисаног реда. Након уписа заглавља, сваки генерисани ред се уписује у датотеку.

Имплементирани део кода се налази у листингу 2.

```

def _write_entity_csv(
    self,
    output_dir: str,
    entity_name: str,
    rows: List[Dict[str, Any]],
) -> None:
    path = os.path.join(output_dir, f"{entity_name}.csv")

    with open(path, "w", newline="", encoding="utf-8") as f:
        if not rows:
            return

        writer = csv.DictWriter(
            f,
            fieldnames=list(rows[0].keys()),
        )

        writer.writeheader()

        for row in rows:
            writer.writerow(row)

```

Листинг 2: Функција за креирање CSV датотеке

Садржај једне CSV датотеке је приказан у следећем исечку. Заглавље одговара пољима ентитета *User*, а сваки ред једној генерисаној инстанци.

```

age,isActive,id,username,email
35,true,443cdef3-6840-4f07-84da-537c1651ddae,adamskayla,daniel06@example.net
21,false,dff4948f-e6b4-4832-96a8-2a951b923e0a,matthew02,matthewcook@example.org

```

6.1.3 Организација излазног директоријума

Поред самог писања CSV датотека, генератор формира и директоријум за резултат генерисања. Назив директоријума одговара називу шеме. Уколико директоријум већ постоји, његово преписивање зависи од вредности опције *overwrite*. Без ове опције систем пријављује грешку, док се са омогућеном опцијом преписивања постојећи директоријум уклања и креира се нови.

Пример за шему *ComprehensiveSystem*, структура резултата ће бити организована на следећи начин:

```

ComprehensiveSystem/
├── User.csv
├── Profile.csv
├── Course.csv
└── generation_details.txt

```

Оваква организација омогућава да се подаци различитих ентитета обрађују независно, што је посебно погодно за даљи увоз или анализу података.

6.1.4 Датотека са подацима о генерисању

Поред CSV датотека, генератор креира и текстуалну датотеку *generation_details.txt*. Ова датотека не садржи тестне податке, већ метаподатке о извршеном генерисању. У њу се уписују назив шеме, коришћена почетна вредност за генератор случајних бројева, време генерисања, излазни формат, број ентитета и укупан број генерисаних записа. За сваки ентитет се додатно наводи број генерисаних записа.

Део имплементације који се односи на генерисање података је приказан у листингу 3.

```

def _write_generation_details(
    self,
    output_dir: str,
    metadata: Dict[str, Any],
    entities: Dict[str, List[Dict[str, Any]]],
) -> None:
    path = os.path.join(output_dir, "generation_details.txt")

    with open(path, "w", encoding="utf-8") as f:
        f.write("TestDataGen CSV Generation\n")
        f.write("=====\n\n")

        f.write(f"Schema: {metadata['schema']}\n")
        f.write(f"Seed: {metadata['seed']}\n")
        f.write(f"Generated at: {metadata['generated_at']}\n\n")

        total = sum(len(rows) for rows in entities.values())

        f.write(f"Format: CSV\n")
        f.write(f"Output directory: {metadata['schema']}\n")
        f.write(f"Entities: {len(entities)}\n")
        f.write(f"Total records: {total}\n\n")

        f.write("Entities\n")
        f.write("-----\n")
        for entity_name, rows in entities.items():
            f.write(f"{entity_name}: {len(rows)} records\n")

```

Листинг 3: Функција која генерише метаподатке за CSV формат

Пример резултата горенаведене функције је дат у наставку:

```

TestDataGen CSV Generation
=====

Schema: ComprehensiveSystem
Seed: 98765
Generated at: 2026-09-02 22:04:29

Format: CSV
Output directory: ComprehensiveSystem
Entities: 3
Total records: 100

Entities
-----
Course: 20 records
Profile: 30 records
User: 50 records

```

Фајл је подељен по секцијама. Прва секција садржи информације о шеми за коју су подаци генерисани, коришћеном *seed*-у и времену генерисања. Друга секција садржи информације о излазном формату, директоријуму у који су резултати смештени, броју ентитета и укупном броју генерисаних записа. Трећа секција приказује број генерисаних записа за сваки ентитет. На тај начин генерисање не садржи само тестне случајеве већ и основне информације које омогућавају лакше праћење начина на који је одређени скуп података настао.

6.1.5 Интеграција са остатком система

Коначно, CSV генератор је интегрисан у постојећи интерфејс командне линије. Функција *generate_csv* формира директоријум за изабрану шему, проверава да ли он већ постоји и, у зависности од операције *overwrite*, креира нови или пријављује постојећи излаз. Након тога креира се инстанца *CSVGenerator* и покреће њена метода *generate*.

На овај начин CSV генератор постаје равноправан део система и може се позвати преко постојећег механизма за избор формата без измене основног процеса парсирања и генерисања тестних података.

6.2 YAML генератор

YAML генератор представља други од три додатна генератора имплементирана у оквиру овог мастер рада. Његова улога је да резултат заједничког процеса генерисања тестних података представи у YAML формату [8]. Као и код осталих генератора, сам поступак избора и комбиновања вредности није део YAML генератора, већ се користи постојећи механизам система. Специфичност овог генератора огледа се у начину припреме и серијализације добијених вредности у хијерархијску YAML структуру.

YAML генератор је имплементиран у оквиру модула *yaml_generator*, а основна класа је *YAMLGenerator*. Класа као улаз прима парсирани *textX* модел спецификације, док се приликом иницијализације могу задавати почетна вредност генератора случајних бројева (енг. *seed*), време генерисања и ниво увлачења (енг. *indent*). Подразумевано се користи *seed* из спецификације уколико друга вредност није експлицитно прослеђена.

Као и CSV генератор, YAML генератор поново користи функције заједничког процеса генерисања из модула *sql_generator*. На тај начин се користе постојећи механизми за прикупљање вредности, њихово комбиновање, обраду експлицитно задатих случајева, одређивање редоследа ентитета и разрешавање референци. Сам генератор је зато усмерен на представљање већ генерисаних података у YAML формату.

6.2.1 Формирање вредности

При серијализацији података потребно је прилагодити вредности типовима које YAML библиотека може да обради. У ту сврху имплементирана је функција *format_value_yaml*. Она прима вредност добијену из заједничког механизма генерисања и враћа YAML-серијализабилну представу.

Функција посебно обрађује *None*, логичке вредности, целе и реалне бројеве, датуме, датум и време, текстуалне вредности и листе. За разлику од CSV генератора, где се *None* представља празним пољем, у YAML генератору се задржава *None*, што се приликом серијализације представља као *YAML null*. Логичке вредности остају логичке вредности. Датум, као и датум и време, представљају се у ISO текстуалном формату. Листе се рекурзивно обрађују тако да њихови елементи задрже одговарајућу структуру.

Кључни део имплементације је наведен у листингу 4.

```
def format_value_yaml(value: Any) -> Any:
    if value is None:
        return None
    if isinstance(value, bool):
        return value
    if isinstance(value, (int, float)):
        return value
    if isinstance(value, datetime):
        return value.isoformat()
    if isinstance(value, date):
        return value.isoformat()
    if isinstance(value, list):
        return [format_value_yaml(v) for v in value]
    if isinstance(value, str):
        return value
    return str(value)
```

Листинг 4: Функција за формирање вредности за *YAML* формат

Оваква имплементација омогућава да се типови вредности не претварају непотребно у текст. На пример, логичке вредности остају логичке вредности, док листа остаје *YAML* секвенца. Тиме се чува структурна природа података приликом њихове серијализације.

6.2.2 Структура *YAML* излаза

Резултујући *YAML* документ организован је у две основне целине: *metadata* и *entities*. Секција *metadata* садржи информације о шеми, коришћеном *seed*-у и времену генерисања. У секцији *entities* налазе се генерисани подаци организовани према називима ентитета. Оваква структура омогућава да се у једном документу обједине метаподаци о генерисању и сами тестни подаци.

Пример структуре *YAML* излаза је:

```
metadata:
  schema: ComprehensiveSystem
  seed: 98765
  generated_at: '2026-09-02 22:04:29'
entities:
  User:
    - id: 00000000-0000-0000-0000-000000000001
      email: admin@system.local
      username: superadmin
      age: 30.0
      status: active
    - id: 00000000-0000-0000-0000-000000000002
      email: kellyriggs@example.net
      username: guest_user
      age: 17.0
      status: pending
    - username: adamsdaniel
      age: 98.0
      status: active
      id: f86c882c-f9af-4a56-8215-c2b76947c104
      email: cody82@example.com
```

У секцији *entities* сваки ентитет представљен је као засебна ставка, чија је вредност листа генерисаних инстанци. Свака инстанца садржи поља ентитета и њихове генерисане вредности. На овај начин структура *YAML* документа директно одражава структуру података добијену из спецификације.

6.2.3 Обрада референци

Посебна пажња у имплементацији посвећена је референцама између ентитета. *YAML* генератор користи исти механизам за разрешавање референци као и остали генератори који раде над подацима типа ентитет. Приликом генерисања вредности референтна поља се повезују са идентификаторима већ генерисаних инстанци одговарајућег ентитета.

Код референци на више инстанци, односно референци облика *ref Entity[]*, вредности се у *YAML* излазу представљају као листа идентификатора, а не као угњеждени објекти. На тај начин се задржавају информације о повезаности ентитета, без непотребног понављања целокупних објеката. Овакав приступ је експлицитно предвиђен у имплементацији *YAML* генератора.

6.2.4 Серијализација *YAML* документа

Након што се подаци генеришу и припреме за *YAML* представу, они се серијализују коришћењем библиотеке *PyYAML* [9]. Увоз библиотеке *yaml* налази се директно у модулу генератора, док је функција *format_value_yaml* задужена за припрему појединачних вредности позвана пре њихове серијализације.

На овај начин *YAML* генератор раздваја два корака: прво се вредности добијене из заједничког механизма прилагођава за *YAML* серијализацију, а затим се целокупна структура записује као *YAML* документ. За разлику од *CSV* генератора, где је потребно ручно формирати табеларни запис и заглавље, *YAML* генератор задржава хијерархијску структуру података.

6.2.5 Интеграција са остатком система

YAML генератор је интегрисан у постојећи механизам командне линије и може се покренути избором формата *yaml*. На тај начин се *YAML* може генерисати самостално или истовремено са осталим подржаним излазним форматима. Документација система наводи *yaml* као један од формата који се могу задати приликом покретања команде *generate*.

Оваква интеграција омогућава да се иста *TestDataGen* спецификација обради једним заједничким поступком, а затим представи у више различитих формата. *YAML* генератор при томе не уводи посебан механизам за генерисање тестних вредности, већ користи постојећи модел и заједнички процес генерисања, док је његова основна одговорност формирање и серијализација *YAML* излаза. Оваква организација је у складу са архитектуром система у којој генератори деле заједнички модел и *pipeline*, а разликују се у начину представљања резултата.

6.3 *Neo4j* генератор

Neo4j генератор представља трећи додатни генератор излазног формата у оквиру овог мастер рада. За разлику од *CSV* и *YAML* генератора чији је основни задатак представљање генерисаних података у другом текстуалном формату, *Neo4j* генератор уводи могућност генерисања података који, поред самих ентитета, садрже и експлицитно дефинисане везе између њих.

Генератор производи *Cypher* скрипту намењену за извршавање у графној бази података *Neo4j* [10]. Генерисани ентитети представљају се као чворови графа, док се везе између ентитета представљају као усмерене везе између одговарајућих чворова. На тај начин је омогућено генерисање тестних графова који поред вредности атрибута, садрже и структуру повезаности између података.

Имплементација *Neo4j* генератора захтевала је проширење више делова система. Поред самог генератора излазног формата, било је потребно проширити граматику језика *TestDataGen* конструкцијом везе (енг. *relationship*), увести механизме генерисања различитих типова веза, проширити валидацију спецификације и додати подршку за постојеће елементе *Visual Studio Code* екстензије.

Као и код осталих генератора, *Neo4j* генератор користи парсирани *textX* модел и заједнички поступак генерисања вредности. На тај начин се генерисање вредности поља ентитета и генерисање вредности

својстава веза између ентитета ослањају на постојеће механизме система, док је специфичност генератора у формирању графичке структуре и њеном записивању у *Cypher* формату.

6.3.1 Проширење граматике језика

Увођење *Neo4j* генератора захтевало је проширење граматике језика *TestDataGen* тако да спецификација, поред ентитета, може да садржи и експлицитно дефинисане везе између њих. У постојећој граматичи *SchemaElement* је проширен тако да може представљати или *Entity* или *Relationship*.

Основна структура конструкције *relationship* дефинисана је на следећи начин:

```
Relationship:
  'relationship' name=ID '{'
    'from' from=[Entity]
    'to' to=[Entity]
    ('properties' '{' properties+=Field '}')?
    ('config' '{' config=RelationshipConfig '}')?
  '}'
;
```

Овом конструкцијом свака веза има назив, полазни ентитет и циљни ентитет. Додатно веза може да садржи блог својстава везе (енг. *properties*) за дефинисање својстава и конфигурациони блок (енг. *config*) за подешавања начина генерисања везе.

Пример везе између ентитета *User* и *Profile* дат је у наставку.

```
relationship UserHasProfile {
  from User
  to Profile
  properties {
    linked_at: datetime {
      boundary
    }
  }
  config {
    strategy: one-to-one
    generate: 30
    include: [
      { linked_at: "2026-01-01T10:00:00" }
    ]
  }
}
```

У горенаведеном примеру *User* представља полазни, а *Profile* циљни ентитет. Веза *UserHasProfile* има једно својство, *linked_at*. Вредност својства се генерише на исти начин као вредност поља ентитета. Конфигурацијом је затим одређено да се користи стратегија *one-to-one*, као и то да је потребно генерисати највише 30 веза и укључити једну кориснички дефинисану вредност за својство.

Грамматика омогућава да својства веза користе постојеће типове и ограничења који се користе и за поља ентитета. Истовремено, референтни типови нису дозвољени као својства веза, јер се повезивање чворова одређује само конструкцијом *from* и *to*. Ово правило се проверава у фази валидације модела.

Конфигурацијом се одређује начин генерисања везе. То подразумева контролу броја и структуре генерисаних веза. Грамматика подржава следеће опције у оквиру конфигурације:

```
RelationshipConfigOption:
  GenerateOption |
  RelationshipStrategyOption |
  MinDegreeOption |
```

```

    MaxDegreeOption |
    IncludeOption
;

RelationshipStrategyOption:
    'strategy:' strategy=RelationshipStrategy
;

RelationshipStrategy:
    'one-to-one'
    | 'one-to-many'
    | 'many-to-one'
    | 'many-to-many'
;

MinDegreeOption:
    'minDegree:' value=INT
;

MaxDegreeOption:
    'maxDegree:' value=INT
;

```

Опција *GenerateOption* се односи на горњу границу броја веза које треба генерисати. Опција *RelationshipStrategyOption* се односи на врсту стратегије при повезивању. Могуће опције су *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many*. Опције за минималан и максималан степен чвора (енг. *MinDegreeOption* и *MaxDegreeOption*) дефинишу доњу и горњу границу броја веза коју треба генерисати за неки чвор. Опција *IncludeOption* омогућава да се укључе експлицитне вредности својстава уколико су претходно дефинисана за неку везу, идентично као код ентитета.

6.3.2 Семантичка анализа

Поред синтаксне провере коју обезбеђује *textX*, у модулу *grammar_louder* додатно се проверава и семантика. За потребе валидације семантике су додате две функције *validate_relationship* и *validate_relationship_config*.

Семантичка анализа подразумева:

- проверу да ли веза има назив
- проверу да ли су дефинисане полазни и циљни ентитет
- провера својстава
 - да ли постоје дупликати својстава
 - да својство не може да буде референтни објекат или листа референтних објеката
- провера конфигурације
 - максималан број генерисаних веза мора бити позитиван број
 - минимални и максимални степен чвора не смеју бити негативне вредности
 - ако су минимални и максимални степен наведени, минималан степен не сме бити већи од максималног степена чвора
 - за стратегију *one-to-one* није дозвољено дефинисање минималног и максималног степена чвора (код *one-to-one* стратегије је то имплицитно јер подразумева да се једна инстанца упарује са највише једном инстанцом другог ентитета).

Овакав принцип валидације је значајан јер спречава да се спецификације са неисправним конфигурацијама проследе генератору. На тај начин се грешке у спецификацији откривају пре самог генерисања излазне *Cypher* скрипте.

6.3.3 Генерисање чвора

У *Neo4j* модулу сваки ентитет представља тип чвора, док свака његова генерисана инстанца представља конкретан чвор. Приликом формирања чвора користе се генерисане вредности поља ентитета.

Специфичност овог корака је у томе што се референтна поља ентитета не записују као својство чвора. Улога референтних поља је да представљају повезаност између ентитета, док се код *Neo4j* генератора та повезаност моделује експлицитним *relationship* конструкцијама. На тај начин се референтна поља не додају као својства чвора, него се њихова семантика представља одговарајућим графним везама.

Кључни део имплементације је приказан у листингу 5.

```
def _build_nodes(self, entity, rows):
    for row in rows:
        properties = {}

        for field in entity.fields:
            if _is_simple_ref(field) or _is_array_ref(field):
                continue

            properties[field.name] = row.get(field.name)

    yield {
        "label": entity.name,
        "properties": properties,
    }
```

Листинг 5: Функција за креирање чворова

Назив ентитета се користи као ознака (енг. *label*) чвора, док се вредности његових поља смештају у својства чвора. Референтна поља се прескачу, јер се везе које представљају повезаност између ентитета касније обрађују као експлицитне графне везе.

Пре генерисања веза неопходно је обезбедити идентификаторе генерисаних инстанци. Генератор зато памти идентификаторе сваког ентитета. При избору идентификатора предност имају поље *id*, затим поље типа *uuid*, а уколико такво поље не постоји користи се прво дефинисано поље ентитета.

Ови идентификатори се касније користе за одређивање конкретних полазних и циљних чворова приликом креирања веза.

6.3.4 Генерисање својстава везе

Поред самог повезивања чворова, *TestDataGen* омогућава да и везе имају сопствена својства. Она се дефинишу у блоку *properties* унутар конструкције *relationship* [11]. Својства веза обрађују се кроз основни механизам генерисања вредности који се користи за поља ентитета. На тај начин и за својства веза могу да се користе подржани типови, стратегије, ограничења и експлицитно дефинисани тестни случајеви.

Оваква организација омогућава да се, на пример, уз везу између корисника и профила генеришу и додатне информације као што је датум повезивања. Пример можемо видети у наставку:

```
relationship UserHasProfile {
    from User
    to Profile
    properties {
        linked_at: datetime {
            boundary
        }
    }
}
config {
    strategy: one-to-one
```

```

        generate: 30
        include: [
            { linked_at: "2026-01-01T10:00:00" }
        ]
    }
}

```

6.3.5 Стратегије генерисања веза

Један од најзначајнијих специфичности *Neo4j* генератора је подршка различитим кардиналностима односа. У имплементацији су издвојене четири стратегије: *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many*.

Избор стратегије се врши у конфигурационом блоку везе. Метода *build_relationships* прво преузима конфигурацију везе, а затим на основу изабране стратегије, позива се одговарајућа функција генерисања веза. У листингу 6 је приказана метода из модула *neo4j_generator*.

```

def _build_relationships(self, relationship):
    options = _relationship_options(
        getattr(relationship, "config", None)
    )
    strategy = options["strategy"]
    from_obj = getattr(relationship, "from_", getattr(relationship, "from", None))
    from_name = from_obj.name if from_obj else None

    from_rows = self._generated_rows.get(from_name, [])
    to_rows = self._generated_rows.get(relationship.to.name, [])

    if not from_rows or not to_rows:
        return iter(())
    prop_rows = self._generate_relationship_rows(relationship)

    kwargs = dict(
        relationship=relationship,
        from_rows=from_rows,
        to_rows=to_rows,
        prop_rows=prop_rows,
        generate=options["generate"],
        min_degree=options["min_degree"],
        max_degree=options["max_degree"],
        seed=self.seed,
    )

    if strategy == "one-to-one":
        return _one_to_one(**kwargs)
    if strategy == "one-to-many":
        return _one_to_many(**kwargs)
    if strategy == "many-to-one":
        return _many_to_one(**kwargs)
    if strategy == "many-to-many":
        return _many_to_many(**kwargs)
    return _one_to_one(**kwargs)

```

Листинг 6: Функција за генерисање веза

Као што се види, прво се преузимају параметри из модела, затим се генеришу својства и на крају позива одговарајућа стратегија генерисања веза. Уколико стратегија није експлицитно наведена, у тренутној имплементацији, се користи подразумевана *one_to_one* стратегија. Све стратегије су издвојене у фолдер *strategies* у модулу *relationship*.

6.3.5.1 Стратегија *one-to-one*

Код стратегије *one-to-one* свака инстанца полазног ентитета се упарује са највише једном инстанцом циљног ентитета. У имплементацији се број веза одређује као минимум броја изворних и циљних инстанци, додатно ограничено вредношћу *generate* уколико је она задата.

Имплементација је приказана у листингу 7.

```
def _one_to_one(relationship, from_rows, to_rows, prop_rows=None,
               generate=None, min_degree=None, max_degree=None, seed=None,
               ):
    limit = min(len(from_rows), len(to_rows))
    if generate is not None:
        limit = min(limit, generate)

    from_obj = getattr(relationship, "from_", getattr(relationship, "from", None))
    from_name = from_obj.name if from_obj else None
    prop_rows = prop_rows or []

    for i in range(limit):
        properties = prop_rows[i] if i < len(prop_rows) else {}
        yield {
            "from_label": from_name,
            "from_id": from_rows[i]["id"],
            "to_label": relationship.to.name,
            "to_id": to_rows[i]["id"],
            "type": relationship.name.upper(),
            "properties": properties,
        }
```

Листинг 7: Стратегија *one-to-one*

На овај начин се прва инстанца полазног ентитета повезује са првом инстанцом циљног, а друга са другом и тако редом. Због тога је број генерисаних веза ограничен бројем доступних инстанци оба (полазног и циљног) ентитета.

6.3.5.2 Стратегија *one-to-many*

Код стратегије *one-to-many* једна инстанца полазног ентитета може бити повезана са више инстанци циљног ентитета. За сваки полазни чвор се одређује насумично степен повезаности у оквиру задатих граница (користе се *minDegree* и *maxDegree* ако су наведене или ако нису онда се користе подразумеване вредности). Након тога се из циљних инстанци бира одговарајући број различитих инстанци. То је дато у исечку имплементације у листингу 8.

```
for source in from_rows:
    degree = rng.randint(min_degree, max_degree)
    targets = rng.sample(to_rows, degree)

    for target in targets:
        if generate is not None and generated >= generate:
            return
```

Листинг 8: Стратегија *one-to-many*

Коришћење *random.sample* обезбеђује се да се у оквиру једног полазног чвора исти циљни чвор не изабере више пута. Укупан број генерисаних веза додатно се ограничава параметром *generate*, ако је наведен.

6.3.5.3 Стратегија *many-to-one*

Стратегија *many-to-one* представља обрнути случај у односу на *one-to-many*. Свака полазна инстанца добија везу ка изабраној инстанци циљног ентитета. Циљни чвор се бира случајно па због тога више различитих полазних чворова може бити повезано са истим циљним чвором.

Листинг 9 приказије најбитнији исечак ове стратегије.

```
for i, source in enumerate(from_rows[:limit]):
    properties = prop_rows[i] if i < len(prop_rows) else {}
    yield {
        "from_label": from_name,
        "from_id": source["id"],
        "to_label": relationship.to.name,
        "to_id": rng.choice(to_rows)["id"],
        "type": relationship.name.upper(),
        "properties": properties,
    }
```

Листинг 9: Стратегија *many-to-one*

Број генерисаних веза ограничен је бројем полазних инстанци и, уколико је наведена, вредношћу ограничења *generate*.

6.3.5.4 Стратегија *many-to-many*

Стратегија *many-to-many* омогућава да свака полазна инстанца буде упарена са више циљних инстанци. За сваки полазни чвор насумично се бира степен повезаности. Степен повезаности мора бити у границама између ограничења *minDegree* и *maxDegree*. Након тога се из циљних инстанци бира одговарајући број различитих чворова. Исечак имплементације се налази у листингу 10:

```
min_degree = min_degree or 1
max_degree = max_degree or len(to_rows)
max_degree = min(max_degree, len(to_rows))

for source in from_rows:
    degree = rng.randint(min_degree, max_degree)
    targets = rng.sample(to_rows, degree)

    for target in targets:
        if generate is not None and generated >= generate:
            return
```

Листинг 10: Стратегија *many-to-many*

На овај начин је могуће формирати густу мрежу повезаности две групе чворова. Истовремено, параметар *generate* представља горњу границу укупног броја генерисаних веза.

6.3.6 Формирање *Cypher* скрипте

Након што су генерисани чворови и везе, добијена структура се претвара у *Cypher* скрипту. У ту сврху *Neo4j* генератор користи функцију *format_value_neo4j*, која вредности добијене из *Python* модела преводи у одговарајуће *Cypher* литерале.

Функција обрађује различите типове вредности и конвертује их у текстуалну вредност. *None* се представља као *Cypher* вредност *null*, логичке вредности као *true* и *false*, бројеви остају бројеви, датум и датуми и време се записују као текстуалне вредности у *ISO* формату, текст се ставља под наводнике, а листе се рекурзивно претварају у *Cypher* листе. Имплементација методе је дата у листингу 11.

```
def format_value_neo4j(value: Any) -> str:
    if value is None:
        return "null"
    if isinstance(value, bool):
        return "true" if value else "false"
    if isinstance(value, (int, float)):
        return str(value)
    if isinstance(value, datetime):
        return json.dumps(value.isoformat(), ensure_ascii=False)
    if isinstance(value, date):
        return json.dumps(value.isoformat(), ensure_ascii=False)
    if isinstance(value, list):
        return "[" + ", ".join(
            format_value_neo4j(v)
            for v in value
        ) + "]"
    return json.dumps(str(value), ensure_ascii=False)
```

Листинг 11: Функција за формирање вредности за Neo4j формат

На овај начин је омогућено да се генерисане вредности директно користе при формирању *Cypher* израза.

Cypher скрипта се формира у две основне целине. Прва целина садржи дефиниције чворова. Друга целина садржи дефиниције веза.

За сваки чвор генерише се *CREATE* израз. Назив ентитета се користи као ознака чвора. Генерисане вредности својстава служе за формирање својства чвора. Имплементација је дата у листингу 12.

```
for node in data["nodes"]:
    label = node["label"]
    properties = node["properties"]

    props = [
        f"{key}: {format_value_neo4j(value)}"
        for key, value in properties.items()
    ]

    lines.append(
        f"CREATE (:{label} {{{', '.join(props)}}});"
```

Листинг 12: Формирање *CREATE* исказа за сваки чвор

На примеру за ентитет *User* може се добити следећи *Cypher* запис:

```
CREATE (:User {id: "f86c882c-f9af-4a56-8215-c2b76947c104", email: "cody82@example.com",
username: "adamsdaniel", age: 98.0, status: "active"});
```

Након чворова, генератор формира записе веза. За сваку везу најпре се помоћу *MATCH* израза проналазе полазни и циљни чвор на основу идентификатора. Затим се помоћу *CREATE* израза формира усмерена веза, али се назив везе записује великим словима. Део имплементације задужен за креирање веза је приказан у листингу 13.


```

for relationship in data["relationships"]:
    props = [
        f"{key}: {format_value_neo4j(value)}"
        for key, value in relationship['properties'].items()
    ]
    props_str = f" {{{', '.join(props)}}}" if props else ""

    lines.extend([
        (
            f"MATCH (from:{relationship['from_label']}) "
            f"{{{id: {format_value_neo4j(relationship['from_id'])}}}}, "
            f"(to:{relationship['to_label']}) "
            f"{{{id: {format_value_neo4j(relationship['to_id'])}}}})"
        ),
        f"CREATE (from)-[:{relationship['type']}]{{props_str}}->(to);",
        "",
    ])

```

Листинг 13: Формирање *MATCH* и *CREATE* исказа за сваку везу

Резултат овог дела имплементације, може дати следећи излаз:

```

MATCH (from:User {id: "00000000-0000-0000-0000-000000000001"}), (to:Profile {id: "3e488f08-
c673-41bc-8362-59c67f999b44"})
CREATE (from)-[:USERHASPROFILE {linked_at: "2026-01-01T10:00:00"}]->(to);

```

На тај начин у коначној скрипти постоје одвојени делови за чворове и везе које их повезују.

6.3.7 Проширење екстензије

Систем *TestDataGen* садржи екстензију за *Visual Studio Code*. Подржани су: бојење текста (енг. *syntax highlighting*), приказ кратког описа (енг. *hover*), предлог допуне текста, одлазак на дефиницију (енг. *go to definition*) и проналажење референци (енг. *find references*).

Увођење конструкције *relationship* захтевало је и проширење екстензије за подршку језика *TestDataGen* [12]. Поред тога што је конструкција морала бити препозната приликом обраде спецификације, било је потребно омогућити кориснику да лакше користи *Visual Studio Code*. Ово проширење није утицало једино на одлазак на дефиницију јер је та опција потребна само ентитетима који могу бити референцирани.

6.3.7.1 Бојење текста (енг. *syntax highlighting*)

Проширење језика захтевало је допуну правила за синтаксно бојење текста. Нове кључне речи конструкције везе, као и вредности стратегија везе, издвојене су као елементи језика који требају да имају одговарајућу представу. На овај начин корисник приликом писања спецификације визуелно може да разликује кључне речи *relationship*, *from*, *to*, *properties* и називе стратегија од текста који не представља кључне речи.

На слици 5 приказан је пример светле теме за имплементирану екстензију *Visual Studio Code*-а.

```

relationship UserHasProfile {
  from User
  to Profile
  properties {
    linked_at: datetime {
      boundary
    }
  }
  config {
    strategy: one-to-one
    generate: 30
    include: [
      { linked_at: "2026-01-01T10:00:00" }
    ]
  }
}

```

Слика 5: Синтаксно бојење у Visual Studio Code.

6.3.7.2 Приказ кратког описа (енг. *hover*)

У Language Server-у су проширени речници који описују елементе језика. Кључне речи *relationship*, *from*, *to* и *properties* су додате у речник кључних речи. Такође је уведен речних стратегија веза, што је приказано у листингу 14.

```

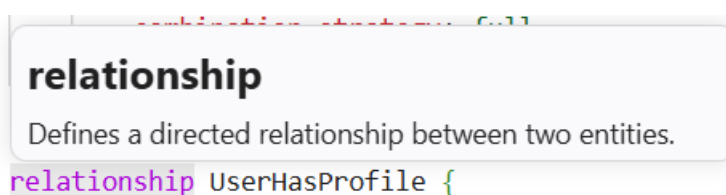
KEYWORDS_DICTIONARY = {
  ...
  "relationship": "Defines a directed relationship between two entities.",
  "properties": "Block containing properties stored on a relationship.",
  "from": "Source entity of the relationship.",
  "to": "Target entity of the relationship.",
  ...
}

RELATIONSHIP_STRATEGY_DICTIONARY = {
  "one-to-one": "Each source entity is connected to exactly one target entity.",
  "one-to-many": "Each source entity may be connected to multiple target entities.",
  "many-to-one": "Multiple source entities may be connected to the same target entity.",
  "many-to-many": "Source and target entities may have multiple connections.",
}

```

Листинг 14: Речници потребни за кратки опис

Речних *RELATIONSHIP_STRATEGY_DICTIONARY* је укључен у *HOVER_DICTIONARY* чиме се обезбеђује да се преласком миша преко кључне речи испише опис новододатих кључних речи и стратегија. На слици 6 приказан је кратки опис конструкта *relationship*.

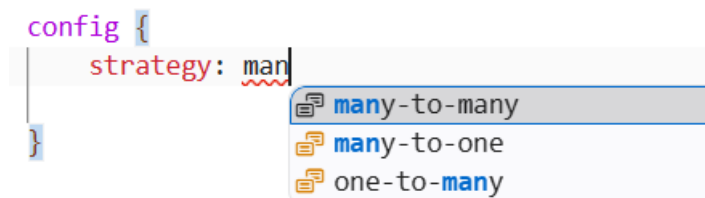


Слика 6: Приказ кратког описа у Visual Studio Code-у.

6.3.7.3 Предлог допуне текста

Поред кључних речи веза, додато је да се предлог допуна текста врши и за стратегије веза. Посебно је значајно поменути да се називи ентитета додају у листу могућих допуна па се након *from* и *to* могу понудити ентитети који су заиста дефинисани у тренутној спецификацији.

Предлог допуна текста за пример стратегије генерисања веза је приказан на слици 7.



Слика 7: Приказ предлога допуне у Visual Studio Code-у.

6.3.7.4 Проналажење референци (енг. *find references*)

За рад са везама, посебно је значајно проналажење референци. Оно омогућава проналажење свих места на којима се одређени ентитет користи. Језички сервер најпре додаје саму дефиницију ентитета, а затим претражује употребу назива ентитета у следећа три контекста (листинг 15):

```
patterns = [
    rf"ref\s+({re.escape(selected_word)})\b",
    rf"from\s+({re.escape(selected_word)})\b",
    rf"to\s+({re.escape(selected_word)})\b",
]
```

Листинг 15: Контексти за проналажење референци

На овај начин се поред постојећих референци типа *ref* обухватају и нове употребе ентитета у конструкцији *relationship*. Пронађене позиције се враћају као *Location* објекти које Visual Studio Code може приказати кориснику.

На слици 8 приказан је пример проналажења референци за ентитет *User*.



Слика 8: Приказ проналаска референци у Visual Studio Code-у.

6.3.8 Интеграција са процесом генерисања

Neo4j генератор не представља независан процес генерисања тестних вредности. Он користи исти поступак за генерисање инстанци ентитета као и остали генератори. У имплементацији се зато поново користе

функције из *sql_generator* модула. То укључује функције за прикупљање вредности, комбиновање, обраду експлицитно задатих случајева, одређивање редоследа ентитета и препознавање референтних поља.

Након генерисања ентитета, *Neo4j* генератор чува генерисане редове и идентификаторе, а затим на основу конструкција *relationship* формирају се везе између већ генерисаних чворова. Метода *build_relationships* преузима полазне и циљне редове, генерише својства веза и прослеђује све потребне параметре изабраној стратегији.

На тај начин је постигнуто раздвајање одговорности. Заједнички механизам одређује вредности тестних података, док је *Neo4j* генератор задужен за њихово организовање у графну структуру и формирање *Cypher* излаза.

6.3.9 Формирање коначне датотеке

На крају процеса генерисања, функција *generate_neo4j* преузима резултат генератора и уписује га у *Cypher* датотеку. Назив датотеке одређује се на основу назива шеме. Имплементација ове методе је дата у листингу 16.

```
def generate_neo4j(model, output_dir, overwrite):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    generator = Neo4JGenerator(
        model,
        timestamp=timestamp,
    )
    cypher = generator.render()
    schema_name = getattr(model, "name", "generated_data")
    file_path = os.path.join(
        output_dir,
        f"{schema_name}.cypher",
    )
    if os.path.exists(file_path) and not overwrite:
        raise FileExistsError(
            f"File {file_path} already exists. Use --overwrite to replace it."
        )
    with open(file_path, "w", encoding="utf-8") as f:
        f.write(cypher)
    return file_path
```

Листинг 16: Функција за креирање *Cypher* датотеке

За шему *ComprehensiveSystem*, на пример, резултат се записује у датотеку:

```
ComprehensiveSystem.cypher
```

Уколико датотека већ постоји, а није прослеђена опција *overwrite* генератор пријављује грешку и не препишује постојећи резултат. На овај начин, понашање *Neo4j* генератора је усклађено са механизмом контроле постојећих излазних датотека.

6.3.10 Пример коначног резултата

Коначан *Cypher* документ садржи најпре записе чворова, а затим веза између њих. Пример садржаја:

```
// --- Node Definitions (records 100) ---
CREATE (:Course {id: "8f8421c8-970c-45ad-86ed-653d21707a11", title: "De-engineered zero-defect parallelism", price: 0, capacity: 10});
CREATE (:Profile {id: "3e488f08-c673-41bc-8362-59c67f999b44", bio: "Dinner check writer into. Focus spring really so. Series western simple hard director in.", birth_date: "1950-01-02", is_verified: false});
CREATE (:User {id: "00000000-0000-0000-0000-000000000001", email: "admin@system.local", username: "superadmin", age: 30.0, status: "active"});
```

```
// --- Relationship Definitions (records 130) ---  
MATCH (from:User {id: "00000000-0000-0000-0000-000000000001"}), (to:Profile {id: "3e488f08-  
c673-41bc-8362-59c67f999b44"})  
CREATE (from)-[:USERHASPROFILE {linked_at: "2026-01-01T10:00:00"}]->(to);
```

Оваква структура директно одговара моделу графне базе: *Course*, *Profile* и *User* представљају чворове, док *USERHASPROFILE* представља усмерену везу између *User*-а и *Profile*-а. Својство *linked_at* припада самој вези, а не неком од чворова.

Пример из мастер рада је покренут у оквиру *Neo4j* базе и резултат за везу *StudentEnrollment* је приказан на слици [9](#).



Слика 9: Приказ везе *StudentEnrollment* из генерисане *Cypher* скрипте.

На овај начин *TestDataGen* може да генерише не само скупове независних тестних записа, него и тестне графове са контролисаном структуром повезаности. То је основна специфичност *Neo4j* генератора у односу на *SQL*, *JSON*, *CSV* и *YAML* генераторе.

6.3.11 Интеграција са командном линијом

Neo4j генератор је интегрисан у постојећи механизам командне линије и може се изабрати навођењем формата *neo4j*. На тај начин се, у оквиру једног покретања система, исти *TestDataGen* модел може користити за генерисање *Neo4j* излаза заједно са другим подржаним форматима.

Пример покретања:

```
testdatagen generate master.tdata --output ./output --format sql,json,report,csv,yaml,neo4j
```

На овај начин се једна спецификација обрађује кроз заједнички процес, а различити генератори преузимају исти концептуални модел и формирају представу прилагођену конкретном излазном формату.

6.3.12 Значај имплементације *Neo4j* генератора

Имплементацијом *Neo4j* генератора *TestDataGen* је проширен са генератора тестних података заснованих првенствено на ентитетима на систем који може да генерише и структуру односа између генерисаних података.

Највећа разлика у односу на остале генераторе је у томе што се приликом генерисања не обрађују само вредности поља, већ и везе између инстанци ентитета. Увођењем конструкције *relationship* омогућено је декларативно описивање графне структуре, док су стратегије *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many* омогућиле различите начине формирања повезаности.

Истовремено, проширење граматике, валидације и *Visual Studio Code* екстензије обезбеђује да нова функционалност буде интегрисана у цео процес рада са језиком *TestDataGen*, од писања спецификације, преко њене провере, до генерисања коначног *Cypher* излаза.

На овај начин је очувана основна архитектура система: опис тестних података остаје одвојен од његове конкретне представе, док се за *Neo4j* формат, поред вредности ентитета, може описати и структура односа која је неопходна за генерисање реалистичних тестних графова.

У оквиру овог мастер рада извршено је проширење система *TestDataGen* са три додатна генератора излазних формата: *CSV*, *YAML* и *Neo4j*. Циљ проширења је био да се омогући коришћење једне *TestDataGen* спецификације за добијање тестних података у различитим излазним форматима, без потребе за изменом структуре, ограничења или стратегије генерисања.

CSV и *YAML* генератори проширују могућност система да представи генерисане тестне податке у табеларном или хијерархијском излазном формату. При њиховој имплементацији, коришћен је постојећи механизам генерисања. Специфичност сваког генератора је реализована у делу који се односи на припрему и записивање података у одговарајући формат. На тај начин је очувано раздвајање процеса генерисања тестних вредности од њихове коначне представе.

Посебан значај у оквиру рада има имплементација *Neo4j* генератора. За разлику од *CSV* и *YAML* генератора, чије је проширење првенствено било усмерено на одређен начин записивања већ генерисаних података, подршка за *Neo4j* захтевала је проширење самог *TestDataGen* језика. Уведена је конструкција *relationship*, којом се омогућава описивање веза између ентитета, као и подршка за њену обраду и генерисање. Имплементиране су различите кардиналности веза, укључујући *one-to-one*, *one-to-many*, *many-to-one* и *many-to-many*, као и могућност дефинисања својстава везе. На основу тако проширене спецификације *Neo4j* генератор формира *Cypher* скрипту којом се могу креирати чворови и везе у графној бази података.

Проширење *relationship*, конструкцијом захтевало је и измене подршке за рад са језиком у развојном окружењу. *Visual Studio Code* екстензија проширена је тако да препознаје нове елементе језика и пружа одговарајућу помоћ при писању спецификације, укључујући бојење текста, допуну речи, приказ описа елемента и проналажење референци. На овај начин нова функционалност није дата само на нивоу генератора, већ интегрисана у целокупан процес *TestDataGen* спецификације.

Реализована проширења задржавају постојећу архитектуру система у којој се процес генерисања тестних података извршава независно од начина њиховог представљања. Оваква организација омогућава да се исти генерисани подаци користе у различитим контекстима, док се за сваки формат примењују правила карактеристична за његову структуру. Додатна подршка за графично представљање података проширује област примене система и показује да се *TestDataGen* може користити не само за генерисање независних података о ентитетима, већ и за генерисање тестних случајева који садрже сложене везе између података.

Као могући кораци даљег развоја издвајају се проширења скупа стратегија за генерисање веза, увођење додатних правила за контролу структуре и густине генерисаних графова, као и даље проширивање подршке коју развојно окружење пружа за нове конструкције језика. На тај начин би се постојећи систем могао даље проширивати без нарушавања основне идеје *TestDataGen* језика - да се опис тестних података одвоји од начина на који се они генеришу и представљају.

Као додатни правац даљег развоја може се издвојити и спровођење тестирања перформанси система. Посебно је значајно испитати понашање генератора *YAML* и *Neo4j* у сценаријима који подразумевају генерисање и обраду великих количина података. То може бити од значаја при употреби ових формата у дистрибуираним системима. Испитивањем времена генерисања, потрошње меморије и других релевантних перформанси могуће је идентификовати потенцијана ограничења система и, уколико се покаже потребним, применити додатне оптимизације у процесу генерисања и записивања података.

Списак слика

Слика 1	Пример разлике конкретног и апстрактног синтаксног стабла.	4
Слика 2	Апстрактна синтакса језика <i>TestDataGen</i>	10
Слика 3	Дијаграм активности тока генерисања тестних података у систему <i>TestDataGen</i>	22
Слика 4	Пример <i>HTML</i> извештаја генерисаног користећи <i>TestDataGen</i>	29
Слика 5	Синтаксно бојење у <i>Visual Studio Code</i>	48
Слика 6	Приказ кратког описа у <i>Visual Studio Code</i> -у.	48
Слика 7	Приказ предлога допуне у <i>Visual Studio Code</i> -у.	49
Слика 8	Приказ проналаска референци у <i>Visual Studio Code</i> -у.	49
Слика 9	Приказ везе <i>StudentEnrollment</i> из генерисане <i>Cypher</i> скрипте.	52

Списак листинга

Листинг 1	Функција за формирање вредности за CSV формат	34
Листинг 2	Функција за креирање CSV датотеке	35
Листинг 3	Функција која генерише метаподатке за CSV формат	36
Листинг 4	Функција за формирање вредности за YAML формат	38
Листинг 5	Функција за креирање чворова	42
Листинг 6	Функција за генерисање веза	43
Листинг 7	Стратегија <i>one-to-one</i>	44
Листинг 8	Стратегија <i>one-to-many</i>	44
Листинг 9	Стратегија <i>many-to-one</i>	45
Листинг 10	Стратегија <i>many-to-many</i>	45
Листинг 11	Функција за формирање вредности за <i>Neo4j</i> формат	46
Листинг 12	Формирање <i>CREATE</i> исказа за сваки чвор	46
Листинг 13	Формирање <i>MATCH</i> и <i>CREATE</i> исказа за сваку везу	47
Листинг 14	Речници потребни за кратки опис	48
Листинг 15	Контексти за проналажење референци	49
Листинг 16	Функција за креирање <i>Cypher</i> датотеке	50

Списак табела

Табела 1	Табела ограничења	18
----------	-------------------------	----

Списак коришћених скраћеница

Скраћеница	Опис
GPL	General-Purpose Languages (језици опште намене)
DSL	Domain-Specific Languages (језици специфични за домен)
SQL	Structured Query Language (структурирани упитни језик)
HTML	HyperText Markup Language (језик за дефинисање садржаја веб странице)
CSS	Cascading Style Sheets (језик за описивање стилова)
UML	Unified Modeling Language (језик за моделовање дијаграма)
AST	Abstract Syntax Tree (апстрактно синтаксно стабло)
CST	Concrete Syntax Tree (конкретно синтаксно стабло)
JSON	JavaScript Object Notation (формат за размену података)
CSV	Comma-separated values (формат за табеларне податке)
YAML	YAML Ain't Markup Language (формат за структуриране податке)
CLI	Command Line Interface (интерфејс командне линије)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)

Списак коришћених појмова

Појам	Објашњење
Софтверски језик	Подгрупа језика чија је намена да омогући комуникацију човек-рачунар или рачунар-рачунар.
Језик опште намене	Подгрупа софтверских језика. Његова примена може да буде у различитим областима и за креирање различитих софтверских апликација.
Домен	Сфера у којој делујемо.
Језик специфичан за домен	Подврста софтверских језика специјализован за један узак домен.
Густина софтверске грешке	Број софтверских грешака на 1000 линија кода.
Језичка какофонија	Ппотреба да програмери познају велики број софтверских језика.
Апстрактна синтакса	Описује концепте из домена, њихове међусобне везе и особине.
Апстрактно синтактно стабло	Апстрактна структурна репрезентација конкретног исказа.
Конкретна синтакса	Интерфејс према крајњем кориснику.
Конкретно синтактно стабло	Конкретна структурна репрезентација конкретног исказа.
Секундарна нотација	Одређена слобода у начину представљања исказа остављена крајњем кориснику.
Семантика	Дефинише смисао исправног језичког исказа.

Биографија

Зовем се Милица Ђумић. Рођена сам 28.10.1999. године у Новом Саду. Основну школу “Славко Родић” сам завршила као Вуковац у Бачком Јарку. Уписала сам гимназију “Исидора Секулић” у Новом Саду, природно-математички смер, из љубави према математици. Завршавам средњу школу као одличан ђак и у току школовања откривам страст према програмирању. Образовање настављам на “Факултету техничких наука” у Новом Саду, на смеру софтверско инжењерство и информационе технологије. Основне студије завршавам у року 2022. године са просеком 8.61 и мало потом рађам ћерку. Едукацију сам наставила после оспособљавања детета за улазак у колектив 2024. године. Усмеравам се на мастер академске студије на “Факултету техничких наука” у области софтверско инжењерство.

Литература

- [1] И. Дејановић, *Језици специфични за домен*. Факултет техничких наука, Нови Сад, 2021.
- [2] M. Mernik, J. Heering, and A. M. Sloane, “When and how to develop domain-specific languages,” *ACM computing surveys (CSUR)*, vol. 37, no. 4, pp. 316–344, 2005.
- [3] B. Perišić, G. Milosavljević, I. Dejanović, and B. Milosavljević, “UML profile for specifying user interfaces of business applications,” *Computer Science and Information Systems*, no. 18, pp. 405–426, 2011.
- [4] I. Dejanović, R. Vadera, G. Milosavljević, and Ž. Vuković, “TextX: A Python tool for Domain-Specific Languages implementation,” *Knowledge-Based Systems*, vol. 115, pp. 1–4, 2017, doi: [10.1016/j.knosys.2016.10.023](https://doi.org/10.1016/j.knosys.2016.10.023).
- [5] P. Ammann, J. Offutt, and J. Offutt, *Introduction to software testing*, vol. 1. Cambridge University Press New York, NY, USA, 2008.
- [6] F. S. Hameed, P. Fatima, M. P. Venkataprasad, J. Subramanian, and S. Ganesan, “Generating classes of test data using fake data generation and static data manipulation,” in *AIP Conference Proceedings*, 2025, p. 20088.
- [7] Y. Shafranovich, “Common format and mime type for comma-separated values (csv) files, 2005,” *Disponivel em:* < <http://www.ietf.org/rfc/rfc4180.txt>, 2008.
- [8] C. Evans, O. Ben-Kiki, and I. döt Net, “YAML Ain’t Markup Language (YAML™) Version 1.2..” 2017.
- [9] M. A. Carcano, “YAML with Yq and Python,” in *DevSecOps and DevOps for Linux: The Foundations*, Springer, 2026, pp. 241–265.
- [10] O. Panzarino, *Learning Cypher: Write powerful and efficient queries for Neo4j with Cypher, its official query language*. Packt Publishing Ltd, 2014.
- [11] I. Robinson, J. Webber, and E. Eifrem, *Graph databases: new opportunities for connected data*. " O'Reilly Media, Inc.", 2015.
- [12] D. Bork and P. Langer, “Language server protocol: An introduction to the protocol, its use, and adoption for web modeling tools,” *Enterprise Modelling and Information Systems Architectures (EMISAJ)-International Journal of Conceptual Modeling*, vol. 18, pp. 9–1, 2023.